



Mémoire PFE de Master

Intelligence Artificielle et Cybersécurité

Réalisé par :

WAFAE BOUAJAJA

Sous le thème :

**Détection des fraudes bancaires par analyse massive
des données et intelligence artificielle**

Encadré par :

Prof. Hajar Filali

Soutenue le 01/10/2026 devant le Jury composé de :

Prof. Honnit Bouchra	Présidente	Faculté des Sciences de Kénitra
Prof. Filali Hajar	Encadrante	Faculté des Sciences de Kénitra
Prof. Ajaj Souad	Examinatrice	Faculté des Sciences de Kénitra
Prof. Kasri Nadia	Examinatrice	Faculté des Sciences de Kénitra

Dédicaces

Ce mémoire est particulièrement dédié

À

Mes chers parents

À

Mes frères et sœurs

À

Tous ceux qui ont contribué de près ou de loin
à la réalisation et à l'aboutissement de ce travail.

Remerciements

*Au terme de ce travail, je tiens à exprimer ma profonde gratitude à Monsieur **Said Tkatek**, coordonnateur du Master, ainsi qu'à Monsieur **Youssef Fakhri**, chef du département, pour leur engagement, leur accompagnement et les efforts déployés en faveur de la qualité de notre formation.*

*J'adresse mes sincères remerciements à mon encadrante, Madame **Hajar Filali**, pour sa disponibilité, ses précieux conseils, son suivi rigoureux ainsi que son soutien tout au long de la réalisation de ce projet.*

*Je remercie également les membres du jury, Madame **Bouchra Honnit**, présidente du jury, ainsi que Madame **Souad Ajjaj** et Madame **Nadia Kasri**, examinatrices, pour l'intérêt qu'elles portent à mon travail et pour l'honneur qu'elles me font en acceptant d'évaluer ce mémoire.*

J'exprime ma reconnaissance à l'ensemble de mes enseignants qui ont contribué à ma formation universitaire et m'ont transmis leurs connaissances, leur savoir-faire et leur expérience tout au long de mon parcours académique.

Enfin, j'adresse mes plus sincères remerciements à mes proches pour leur soutien constant, leur confiance, leurs encouragements et leur présence à chaque étape de mon parcours. Leur appui moral a été une source de motivation essentielle dans l'aboutissement de ce travail.

À toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce mémoire, j'adresse mes plus vifs remerciements.

Résumé

Ce mémoire porte sur la détection des fraudes bancaires par l'analyse massive de données et l'intelligence artificielle. Il s'appuie sur deux datasets réels de détection de fraudes bancaires : un dataset déséquilibré (284 807 transactions, 0,17 % de fraudes) et un dataset équilibré (568 630 transactions, 50 % de fraudes), afin d'évaluer l'impact du déséquilibre de classes sur les performances des modèles. Plusieurs algorithmes d'apprentissage automatique sont entraînés et comparés, notamment les Forêts Aléatoires (Random Forest), les Machines à Vecteurs de Support (SVM), les Réseaux de Neurones Récurrents (RNN) — complétés par une variante LSTM appliquée uniquement au dataset déséquilibré — et l'algorithme de clustering K-Means. Des techniques de rééchantillonnage telles que SMOTE sont appliquées pour traiter le problème de déséquilibre entre les classes. L'évaluation est réalisée sur trois ensembles distincts — entraînement, validation et test — à l'aide de cinq métriques complémentaires : l'Accuracy, la Précision, le Rappel (Recall), le F1-Score et le Taux de Faux Positifs (False Positive Rate). Les résultats montrent que Random Forest offre les meilleures performances sur le dataset équilibré ($F1 = 99,9\%$), tandis que RNN et LSTM se distinguent sur le dataset déséquilibré ($F1 \approx 93\%$), ouvrant la voie à des systèmes de surveillance intelligents déployables à grande échelle dans les institutions financières.

Mots clés :

Détection de fraude bancaire · Apprentissage automatique · Intelligence artificielle · Big Data · Random Forest · SVM · RNN · LSTM · K-Means · SMOTE · F1-Score · Déséquilibre de classes

Abstract

This work focuses on the detection of banking fraud through massive data analysis and artificial intelligence. It relies on two real banking fraud detection datasets : an imbalanced dataset (284,807 transactions, 0.17 % fraud) and a balanced dataset (568,630 transactions, 50 % fraud), in order to evaluate the impact of class imbalance on model performance. Several machine learning algorithms are trained and compared, namely Random Forest, Support Vector Machines (SVM), Recurrent Neural Networks (RNN) — complemented by an LSTM variant applied only to the imbalanced dataset — and the K-Means clustering algorithm. Resampling techniques such as SMOTE are applied to address the class imbalance problem. The evaluation is carried out on three distinct sets — training, validation and test — using five complementary metrics : Accuracy, Precision, Recall, F1-Score and False Positive Rate (FPR). Results show that Random Forest achieves the best performance on the balanced dataset ($F1 = 99.9\%$), while RNN and LSTM stand out on the imbalanced dataset ($F1 \approx 93\%$), paving the way for intelligent monitoring systems deployable at large scale in financial institutions.

Keywords :

Banking fraud detection · Machine learning · Artificial intelligence · Big Data · Random Forest · SVM · RNN · LSTM · K-Means · SMOTE · F1-Score · Class imbalance

Table des matières

Remerciements	I
Résumé	II
Abstract	III
Liste des Figures	VI
Liste des Tableaux	VII
Nomenclatures	IX
Introduction Générale	1
Chapitre 1 : État de l'art	3
1-1 Introduction	3
1-2 Contexte Général	3
1-3 Problématique	4
1-4 Définition de la fraude	4
1-5 Types de fraudes	5
1-5-1 Fraudes liées à l'identité et aux données personnelles	5
1-5-2 Fraudes Financières et aux Investissements	5
1-5-3 Fraudes ciblant les Entreprises	6
1-5-4 Fraudes fiscales et douanières	6
1-5-5 Fraudes internes	7
1-5-6 Fraudes basées sur la manipulation psychologique	8
1-6 Fraudes Bancaires	8
1-6-1 Types de fraudes bancaires	9
1-6-2 Méthodes de détection des fraudes bancaires	10
a. Méthodes traditionnelles	11
b. Méthodes modernes (Machine Learning et Deep Learning)	11
1-6-3 Cadre réglementaire de la lutte contre la fraude bancaire	15
1-7 Travaux connexes	16
1-8 Défis et limites des systèmes actuels de détection par IA	17
1-9 Conclusion	18
Chapitre 2 : Approche Proposée	19
2-1 Introduction	19
2-2 Architecture générale du système proposé	19
2-2-1 Démarche adoptée	19
2-2-2 Pipeline du système proposé	19
2-3 Méthodologie	24
2-3-1 Random Forest	24
2-3-2 Support Vector Machine (SVM)	25

2-3-3 Réseaux de Neurones Récurrents (RNN)	27
2-3-4 K-Means	28
2-3-5 SMOTE (Synthetic Minority Over-sampling Technique)	29
2-3-6 Pondération des classes (class_weight='balanced')	30
2-3-7 Sous-échantillonnage aléatoire (undersampling)	31
2-4 Prétraitement et normalisation	32
2-5 Extraction des caractéristiques	32
2-6 Classification	34
2-6-1 Division des données	34
2-6-2 Règle de décision et seuil de classification	34
2-7 Conclusion	35
Chapitre 3 : Résultats et Discussion	37
3-1 Introduction	37
3-2 Configuration expérimentale	37
3-2-1 Environnement de travail	37
3-2-2 Détails d'implémentation	38
3-3 Base de données	39
3-3-1 Dataset déséquilibré (creditcard.csv)	39
3-3-2 Dataset équilibré (creditcard2023.csv)	39
3-3-3 Synthèse comparative	40
3-4 Métriques d'évaluation	42
3-4-1 Accuracy (Exactitude)	42
3-4-2 Précision (Precision)	42
3-4-3 Rappel (Recall)	43
3-4-4 F1-Score	43
3-4-5 Taux de Faux Positifs (False Positive Rate)	43
3-4-6 Silhouette Score (spécifique à K-Means)	43
3-5 Analyse des performances	45
3-5-1 Résultats sur le dataset déséquilibré (creditcard)	45
3-5-2 Résultats sur le dataset équilibré (creditcard2023)	47
3-5-3 Comparaison croisée et discussion	49
3-6 Étude comparative avec les travaux connexes	50
3-7 Conclusion	50
Conclusion Générale et Perspectives	52

Liste des Figures

Figure 1.1 : Les trois facettes du triangle de la fraude	10
Figure 1.2 : Les principales sources de fraude en valeur (%)	12
Figure 1.3 : Processus d'une attaque de Phishing	13
Figure 1.4 : Architecture du Random Forest	15
Figure 1.5 : Principe des SVM.....	15
Figure 1.6 : Étapes de l'algorithme K-Means.....	16
Figure 1.7 : Architecture du RNN comparée au FNN	16
Figure 2.1 : Pipeline global du système de détection proposé.....	21

Liste des Tableaux

Tableau 1.1 : Récapitulatif des principaux types de fraudes	11
Tableau 1.2 : Récapitulatif des types de fraudes bancaires	13
Tableau 1.3 : Comparaison entre méthodes traditionnelles et modernes	15
Tableau 1.4 : Comparaison entre la DSP2/PSD2 et le RGPD	16
Tableau 2.1 : Paramètres du modèle Random Forest	23
Tableau 2.2 : Paramètres du modèle SVM	25
Tableau 2.3 : Paramètres des modèles RNN et LSTM	27
Tableau 2.4 : Paramètres du modèle K-Means	29
Tableau 2.5 : Structure comparée des deux datasets	30
Tableau 3.1 : Bibliothèques Python utilisées	37
Tableau 3.2 : Répartition des classes – dataset déséquilibré	39
Tableau 3.3 : Répartition des classes – dataset équilibré	40
Tableau 3.4 : Synthèse comparative des deux datasets	40
Tableau 3.5 : Résultats comparatifs sur le dataset déséquilibré	44
Tableau 3.6 : Matrice de confusion – Random Forest (déséquilibré)	44
Tableau 3.7 : Matrice de confusion – SVM (déséquilibré)	44
Tableau 3.8 : Matrice de confusion – K-Means (déséquilibré)	45
Tableau 3.9 : Matrice de confusion – RNN (déséquilibré)	45
Tableau 3.10 : Matrice de confusion – LSTM (déséquilibré)	45
Tableau 3.11 : Résultats comparatifs sur le dataset équilibré	45
Tableau 3.12 : Matrice de confusion – Random Forest (équilibré)	46
Tableau 3.13 : Matrice de confusion – SVM (équilibré)	46

Tableau 3.14 : Matrice de confusion – K-Means (équilibré)	46
Tableau 3.15 : Matrice de confusion – RNN (équilibré)	47
Tableau 3.16 : Comparaison avec les travaux connexes	47

Nomenclatures

IA	Intelligence Artificielle
ML	Machine Learning (Apprentissage Automatique)
DL	Deep Learning (Apprentissage Profond)
RF	Random Forest (Forêt Aléatoire)
SVM	Support Vector Machine (Machine à Vecteurs de Support)
RNN	Recurrent Neural Network (Réseau de Neurones Récurrent)
LSTM	Long Short-Term Memory
PCA	Principal Component Analysis (Analyse en Composantes Principales)
SMOTE	Synthetic Minority Over-sampling Technique
BPTT	Backpropagation Through Time (Rétropropagation à travers le temps)
VP	Vrais Positifs
VN	Vrais Négatifs
FP	Faux Positifs
FN	Faux Négatifs
FPR	False Positive Rate (Taux de Faux Positifs)
GPU	Graphics Processing Unit
CPU	Central Processing Unit
PFE	Projet de Fin d'Études
ACFE	Association of Certified Fraud Examiners
IBAN	International Bank Account Number
RIB	Relevé d'Identité Bancaire
QR code	Quick Response code

Introduction Générale

Avec l'essor rapide des technologies numériques et la généralisation des services bancaires en ligne, les transactions financières électroniques occupent aujourd'hui une place centrale dans l'économie mondiale. Cette transformation numérique, bien qu'elle offre de nombreux avantages en termes de rapidité et d'accessibilité, expose également les institutions financières à des risques croissants de fraude. Les fraudeurs exploitent les failles des systèmes informatiques et adoptent des techniques de plus en plus sophistiquées, rendant leur détection particulièrement difficile.

Face à ce constat, la détection des fraudes bancaires est devenue un enjeu majeur pour les banques, les organismes de régulation et les entreprises de technologie financière. Les approches traditionnelles basées sur des règles fixes et des contrôles manuels ont rapidement montré leurs limites face au volume massif de données générées quotidiennement et à l'évolution constante des comportements frauduleux. Il devient donc impératif de développer des systèmes automatisés, intelligents et capables de s'adapter en temps réel aux nouvelles menaces.

C'est dans ce contexte que l'intelligence artificielle et l'analyse massive de données (Big Data) s'imposent comme des solutions prometteuses. Le Machine Learning et le Deep Learning permettent en effet d'extraire des connaissances pertinentes à partir de grandes quantités de données, d'identifier des schémas complexes et de détecter automatiquement les transactions suspectes avec une précision élevée.

Ce mémoire s'intéresse à la problématique de la détection des fraudes bancaires par l'analyse massive de données et l'intelligence artificielle. Il s'appuie sur deux datasets réels de transactions bancaires — un dataset déséquilibré reflétant les conditions réelles d'un système bancaire, et un dataset équilibré permettant d'évaluer les modèles dans des conditions idéales — pour entraîner et comparer plusieurs modèles d'apprentissage automatique : Random Forest, SVM, RNN, LSTM et K-Means. L'évaluation est réalisée sur trois ensembles distincts — entraînement, validation et test — à l'aide de cinq métriques complémentaires : Accuracy, Précision, Recall, F1-Score et False Positive Rate (FPR).

Ce mémoire est organisé de la manière suivante :

- Le Chapitre 1 présente l'état de l'art sur la fraude bancaire, ses différents types, ainsi que les méthodes de détection existantes, des approches traditionnelles aux techniques modernes basées sur le Machine Learning et le Deep Learning.
- Le Chapitre 2 présente l'approche proposée dans ce mémoire : l'architecture générale du système, la méthodologie détaillée de chaque algorithme utilisé (Random Forest, SVM, RNN, K-Means), l'extraction des caractéristiques et la phase de classification.

- Le Chapitre 3 décrit les résultats expérimentaux obtenus sur les deux datasets et propose une analyse comparative des différents modèles.

Ce mémoire se termine par une Conclusion Générale qui synthétise les principaux résultats obtenus, discute des limites de l'approche adoptée et propose des perspectives d'amélioration pour les travaux futurs.

Chapitre 1 : État de l'art

1-1 Introduction

La fraude bancaire constitue aujourd'hui l'une des menaces les plus importantes auxquelles font face les institutions financières à l'échelle mondiale. Avec la généralisation des services bancaires numériques et l'augmentation exponentielle des transactions en ligne, les fraudeurs disposent d'un terrain fertile pour développer des techniques d'escroquerie toujours plus sophistiquées et difficiles à détecter. Afin

de mieux appréhender ce phénomène et de poser les bases théoriques nécessaires à la réalisation de ce mémoire, ce chapitre propose un état de l'art complet sur la fraude bancaire et les méthodes permettant de la détecter. Nous commençons par situer le contexte général de la fraude bancaire à l'ère du numérique,

avant de formuler la problématique centrale de ce travail, à savoir la détection d'événements rares dans un flux massif de transactions. La notion de fraude est ensuite définie et ses fondements sont présentés à travers le triangle de la fraude, qui met en évidence les trois facteurs déterminants dans la commission d'un acte frauduleux : la pression, l'opportunité et la rationalisation. Nous dressons

par la suite une typologie des principales formes de fraude, avant de nous recentrer spécifiquement sur les fraudes bancaires, en détaillant leurs différents types, leurs mécanismes de fonctionnement ainsi que les méthodes permettant de les détecter, des approches traditionnelles basées sur des règles fixes aux méthodes modernes fondées sur le Machine Learning et le Deep Learning (Random Forest, SVM, K-Means, RNN/LSTM), qui constituent le cœur du travail expérimental de ce mémoire. Le cadre réglementaire encadrant la lutte contre la fraude bancaire est également abordé. Ce chapitre se termine

par une revue de travaux connexes récents, qui permet de situer ce mémoire par rapport à l'état de la recherche, ainsi que par une discussion des principaux défis et limites que rencontrent encore aujourd'hui les systèmes de détection basés sur l'IA.

1-2 Contexte Général

Avec la digitalisation croissante des services financiers, les banques et institutions financières traitent aujourd'hui des millions de transactions chaque jour. Les paiements en ligne, les virements instantanés et les applications mobiles ont transformé le rapport à l'argent, rendant les échanges financiers plus rapides et plus accessibles. Mais cette évolution a aussi une contrepartie : elle génère un volume colossal de données et ouvre la porte à de nouvelles formes de fraude de plus en plus sophistiquées. Les fraudeurs

s'adaptent continuellement à ces nouvelles technologies. Vol de données, phishing, usurpation d'identité, fraude à la carte bancaire — les techniques se multiplient et deviennent de plus en plus difficiles à

détecter, menaçant aussi bien les particuliers que les institutions financières à l'échelle mondiale. Face à ce constat, les méthodes traditionnelles de détection, basées sur des règles fixes et des contrôles manuels, atteignent rapidement leurs limites. Il est tout simplement impossible pour un analyste humain de surveiller manuellement des millions de transactions générées en temps réel. À titre d'exemple, le premier dataset utilisé dans ce mémoire contient 284 807 transactions réelles, dont seulement 492 sont frauduleuses — ce qui illustre à quel point ces actes sont difficiles à identifier parmi un volume massif de données. C'est précisément là qu'interviennent le Big Data et l'intelligence artificielle. Cette masse de données transactionnelles, loin d'être un obstacle, devient une ressource précieuse dès lors qu'elle est correctement exploitée. Le Machine Learning permet d'apprendre à partir de l'historique des transactions, d'identifier automatiquement les comportements suspects et de s'adapter en temps réel aux nouvelles formes de fraude. C'est exactement dans cette perspective que s'inscrit ce mémoire.

1-3 Problématique

La détection des fraudes bancaires par intelligence artificielle soulève un défi fondamental : dans la réalité, les fraudes sont des événements extrêmement rares. Sur 284 807 transactions réelles, seulement 492 sont frauduleuses, soit à peine 0,17 %. Cette rareté crée un déséquilibre extrême entre les classes qui pose un vrai problème pour les algorithmes d'apprentissage automatique — un modèle peut atteindre 99 % d'accuracy en prédisant systématiquement "transaction légitime", sans détecter la moindre fraude. Ce constat soulève une question centrale : *les algorithmes de Machine Learning et de Deep Learning sont-ils capables de détecter efficacement des fraudes aussi rares, et comment se comportent-ils face à ce déséquilibre ?*

Pour répondre à cette question, ce mémoire s'appuie sur deux configurations de données : un dataset réel déséquilibré, qui reflète les conditions réelles d'un système bancaire, et un dataset équilibré, qui permet d'évaluer les performances des modèles dans des conditions idéales. Cette double expérimentation permet d'observer concrètement l'impact du déséquilibre sur les performances de cinq algorithmes — Random Forest, SVM, K-Means, RNN et LSTM — et d'identifier lequel offre le meilleur compromis entre détection des fraudes et minimisation des fausses alertes.

1-4 Définition de la fraude

La fraude est définie comme un acte réalisé en utilisant des moyens déloyaux destinés à surprendre un consentement, à obtenir un avantage matériel ou moral indu, ou réalisé avec l'intention d'échapper à l'exécution des lois [1].

Selon l'Association of Certified Fraud Examiners (ACFE), la fraude correspond à l'utilisation intentionnelle de la tromperie pour obtenir un gain injuste ou illégal.

La fraude repose généralement sur trois facteurs interdépendants, illustrés par le triangle de la fraude (voir Figure 1.1) : la **pression** financière ou professionnelle qui crée la motivation, l'**opportunité** offerte

par un dispositif de contrôle absent ou insuffisant, et la **rationalisation** par laquelle le fraudeur justifie son acte [1].



Figure 1.1 : Les trois facettes du triangle de la fraude

1-5 Types de fraudes

Les formes de fraude sont nombreuses et ne cessent d'évoluer. Elles peuvent toucher les particuliers comme les entreprises et les administrations [2]. Voici un aperçu des principales catégories de fraudes :

1-5-1 Fraudes liées à l'identité et aux données personnelles

- **Fraude Documentaire** : utilisation de documents contrefaits, falsifiés ou obtenus frauduleusement (faux diplômes pour un recrutement, fausses pièces justificatives pour un logement social, faux documents d'état civil).
- **Usurpation d'Identité** : utilisation des éléments d'identification d'une autre personne pour accéder à des droits ou commettre une infraction (dépôt de demande de titres, accès aux prestations, etc.).

1-5-2 Fraudes Financières et aux Investissements

- **Fraude à la carte de crédit/débit** : utilisation non autorisée des informations d'une carte de paiement pour des achats ou retraits, notamment les achats en ligne après vol des numéros de carte (fraude *Card-Not-Present*).

- **Systèmes Pyramidaux et Ponzi** : promettre des rendements élevés aux investisseurs en utilisant les fonds des nouveaux entrants pour payer les anciens, sans activité économique réelle.
- **Faux placements** : inciter à investir dans des produits frauduleux (crypto-actifs, vin, diamants, formations en ligne) avec la promesse de gains rapides et importants.

1-5-3 Fraudes ciblant les Entreprises

- **Fraude au Faux Fournisseur/Faux RIB** : un fraudeur se fait passer pour un fournisseur habituel et demande un changement de coordonnées bancaires (RIB/IBAN) pour détourner les paiements vers son compte.
- **Fraude au Président** : l'escroc se fait passer pour le dirigeant de l'entreprise et demande à un employé d'effectuer un virement urgent et confidentiel, souvent vers l'étranger, sous un faux prétexte.

1-5-4 Fraudes fiscales et douanières

- **Fraude fiscale** : action délibérée et illégale d'un contribuable conduisant à réduire le montant des impôts qu'il verse au trésor public, notamment en ne déclarant pas ou en sous-évaluant ses revenus, ses activités économiques ou des éléments de son patrimoine [3].
- **Distinction avec les erreurs involontaires** : les erreurs et omissions involontaires tiennent souvent à la complexité du droit fiscal, source d'interprétations divergentes, et doivent néanmoins donner lieu à des redressements.
- **Distinction avec l'optimisation fiscale** : l'optimisation fiscale consiste à utiliser les failles de la législation pour minorer l'impôt dû sans pour autant que ce soit illégal.
- **Évasion fiscale** : zone grise entre la fraude et l'optimisation, qui consiste par exemple à transférer des bénéfices dans des pays où ils sont moins imposés en jouant sur les ambiguïtés de la définition de prix de transfert aux conditions du marché.
- **Constat de la Cour des comptes** : en pratique, la distinction est souvent difficile ; un chapitre du rapport public annuel de 2010 montrait que la majoration de 40 % pour « manquement délibéré », qui marque la frontière entre la répression de la fraude et la rectification d'erreurs involontaires, est parfois appliquée sans référence à une doctrine claire. Dans un rapport de décembre 2025, elle souligne de nouveau que « La DGFIP gagnerait à mieux formaliser sa doctrine d'application des sanctions administratives et à s'assurer de sa bonne application ».
- **Fraude douanière** : les infractions douanières occupent une place singulière dans le paysage répressif, à la frontière du droit pénal et du droit fiscal ; elles constituent un contentieux technique et autonome, régi principalement par le Code des douanes, dont la spécificité tient à la fois à la nature des comportements incriminés, aux pouvoirs étendus reconnus à l'administration des douanes et au régime procédural dérogatoire qui les accompagne [4].
- **Sanctions applicables** : longtemps perçues comme de simples manquements administratifs,

les infractions douanières revêtent aujourd'hui une dimension pénale affirmée, pouvant entraîner des sanctions extrêmement lourdes : amendes proportionnelles, confiscations, peines d'emprisonnement, interdictions professionnelles, ainsi que des atteintes durables au patrimoine et à la réputation des personnes poursuivies.

1-5-5 Fraudes internes

- **Fraude interne** : également appelée fraude professionnelle, désigne les activités frauduleuses commises par les employés d'une organisation contre celle-ci. Tous les employés, mais aussi les prestataires, peuvent potentiellement être impliqués dans des activités frauduleuses, quel que soit leur niveau hiérarchique ou leurs responsabilités. Ce type de fraude se manifeste généralement par un détournement de fonds ou de biens, mais inclut également des comportements visant à tromper l'entreprise. Lorsque les employés ont accès aux systèmes et aux informations internes, il existe un risque qu'ils les utilisent à mauvais escient et exploitent les faiblesses de l'organisation. Bien que la majorité des employés agissent avec intégrité, la fraude interne est une réalité qui peut affecter toutes les entreprises, entraînant des pertes chaque année. L'ACFE (Association of Certified Fraud Examiners) estime que les organisations perdent chaque année 5 % de leur chiffre d'affaires à cause de la fraude [5].
- **Fraude externe** : en revanche, la fraude externe est commise par des individus ou des groupes extérieurs à l'organisation. Ces acteurs externes peuvent utiliser des techniques comme l'ingénierie sociale pour accéder à des informations sensibles ou à des ressources financières.
- **Types de fraude interne** : la fraude interne peut avoir un impact énorme sur une organisation et être difficile à surmonter, en particulier pour les petites entreprises. À ce propos, on peut distinguer plusieurs types de fraudes aux impacts très différents : détournement d'actifs (stocks, matériel, données), manipulation des transactions financières (faux fournisseurs, double paiement, coordonnées bancaires modifiées), abus sur les notes de frais, falsification de justificatifs, ou encore contournement du contrôle des opérations via des validations complaisantes. Une bonne gestion passe par un contrôle interne renforcé : séparation des tâches, revue des droits d'accès, rapprochements et contrôles de cohérence, traçabilité des validations et vérifications ciblées sur les postes sensibles. Documenter ces scénarios aide aussi à comparer chaque cas de fraude à ceux observés dans d'autres entreprises, à identifier les failles récurrentes et à ajuster les processus de validation, d'achat et de paiement de manière pragmatique.
- **Pertes financières** : elles entraînent une diminution de la trésorerie, affectant la capacité de l'entreprise à payer ses employés et ses fournisseurs. De plus, l'organisation peut avoir à supporter des coûts importants liés à des actions en justice, à des enquêtes ou au paiement de pénalités et d'amendes contractuelles.
- **Dommages à la réputation** : ils sont parfois plus difficiles à réparer qu'une perte financière. Une fois que la mauvaise publicité est diffusée, il est difficile de regagner la confiance des clients et des employés. Un exemple frappant est la chute du cours des actions dès que toute forme de fraude interne est révélée.

- **Interruption des opérations commerciales** : elle peut créer un environnement de travail instable, empêchant l'entreprise de générer de la valeur comme prévu. Des ressources doivent être investies pour restaurer les capacités de production affectées par la fraude, ce qui entraîne des coûts supplémentaires.
- **Moral des employés** : il peut également décliner : les entreprises dépendent d'employés motivés, et la méfiance envers leur employeur ou leurs collègues peut réduire l'engagement et accroître le désir de quitter l'entreprise.

1-5-6 Fraudes basées sur la manipulation psychologique

- **Arnaque aux sentiments (Romance Scam)** : l'escroc développe une relation émotionnelle avec la victime, souvent sur des sites de rencontre, avant de lui demander de l'argent sous de faux prétextes.
- **Loterie ou Gains frauduleux** : informer la victime qu'elle a gagné un prix important mais qu'elle doit payer des frais (taxes, frais de dossier) pour le recevoir.
- **Faux ordres de paiement** : convaincre la victime d'autoriser un paiement vers un compte frauduleux sous le prétexte d'une urgence ou d'une fausse annulation d'opération.

Tableau 1.1 : Récapitulatif des principaux types de fraudes [2].

Type de fraude	Description	Exemples
Fraudes liées à l'identité	Utilisation frauduleuse des données personnelles d'une personne	Fraude documentaire, usurpation d'identité
Fraudes financières	Manipulation des systèmes financiers pour obtenir un gain illégal	Fraude à la carte, systèmes Ponzi, faux placements
Fraudes ciblant les entreprises	Escroqueries visant les organisations et leurs employés	Fraude au président, faux fournisseur
Fraudes psychologiques	Manipulation émotionnelle pour soutirer de l'argent	Romance scam, loterie frauduleuse

1-6 Fraudes Bancaires

Les fraudes bancaires se multiplient, avec des méthodes devenant chaque jour plus élaborées : arnaques à la carte, fraude au virement, piratage de compte, prélèvements frauduleux, escroqueries. Les arnaqueurs, toujours plus inventifs, usurpent l'identité d'une banque ou d'autres organismes de confiance pour soutirer des informations sensibles et réaliser des opérations frauduleuses [6].

La figure 1.2 illustre la répartition des principales sources de fraude en valeur. On constate que le paiement par carte représente la part la plus importante avec **38 %**, suivi du chèque (**31 %**) et du virement

(26 %), ce qui confirme que les fraudes financières constituent la menace la plus répandue dans le secteur bancaire [15].

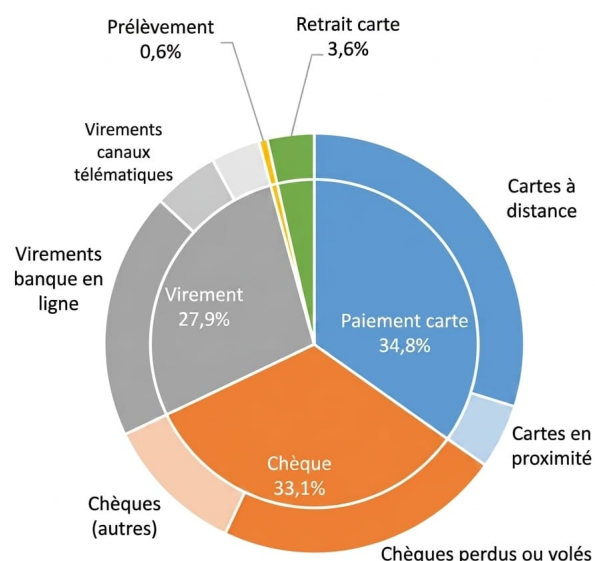


Figure 1.2 : Les principales sources de fraude en valeur (%)

1-6-1 Types de fraudes bancaires

- **La fraude au RIB :** un escroc se fait passer pour un créancier (bailleur, fournisseur, opérateur) et demande d’effectuer les virements sur un nouveau compte bancaire. Une fois le virement effectué, l’argent est perdu.
- **La fraude à la fausse carte bancaire avec QR code :** un escroc adresse un courrier comprenant une fausse carte bancaire accompagnée d’un QR code d’activation. Le scan de ce code redirige vers un faux site bancaire afin de récupérer les données bancaires de la victime.
- **Les faux livrets et placements :** les fraudeurs proposent des placements ou livrets très rémunérateurs, parfois via des réseaux sociaux ou des e-mails, en usurpant l’identité d’une banque pour paraître crédibles.
- **La fraude au prêt ou rachat de crédit :** la victime reçoit une offre de prêt à un taux trop avantageux. Les escrocs collectent les données personnelles et bancaires pour contracter un crédit au nom de la victime.
- **La fraude au conseiller bancaire :** l’escroc se fait passer pour un conseiller bancaire et demande les identifiants, codes de carte bancaire ou mots de passe sous prétexte de vérifier le compte.
- **Les autres fraudes courantes :**
 - *Phishing (hameçonnage) :* e-mails ou SMS imitant la banque pour récupérer les identifiants de connexion.

La figure 1.3 illustre les étapes d'une attaque de phishing, technique très répandue dans les fraudes bancaires [16, 15].

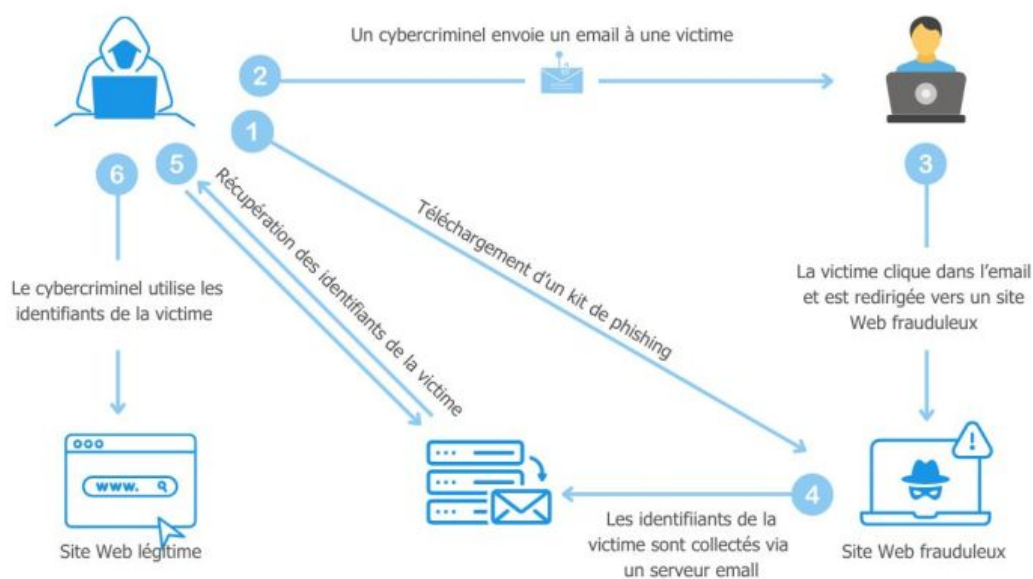


Figure 1.3 : Processus d'une attaque de Phishing

- *Vol de carte bancaire* : retrait ou paiement non autorisé avec les informations de la carte.
- *Fraude aux chèques* : utilisation de faux chèques.

Tableau 1.2 : Récapitulatif des types de fraudes bancaires [6].

Type de fraude	Description	Risque
Fraude au RIB	Détournement de virements vers un compte frauduleux	Perte financière totale et irréversible
Fausse carte avec QR code	Récupération des données bancaires via un faux site	Vol des identifiants bancaires
Faux livrets et placements	Proposition de placements fictifs très rémunérateurs	Perte des fonds investis
Fraude au prêt	Offre de crédit frauduleuse pour collecter des données	Usurpation d'identité et endettement
Fraude au conseiller bancaire	Usurpation d'identité du conseiller	Accès aux comptes et vol d'argent
Phishing	E-mails ou SMS imitant la banque	Vol des identifiants de connexion

1-6-2 Méthodes de détection des fraudes bancaires

La détection des fraudes bancaires repose sur différentes approches qui ont évolué au fil du temps. Les

premières méthodes étaient principalement basées sur des règles fixes et des analyses statistiques, tandis que les approches modernes exploitent l'intelligence artificielle, notamment le Machine Learning et le Deep Learning [8, 14].

Tableau 1.3 : Comparaison entre méthodes traditionnelles et modernes [7, 13].

Critère	Méthodes traditionnelles	Méthodes modernes (ML/DL)
Approche	Basée sur des règles fixes et des seuils prédéfinis	Basée sur l'apprentissage automatique à partir des données
Exemples	Règles IF-THEN, analyse statistique simple	Random Forest, SVM, Réseaux de neurones
Adaptabilité	Faible (ne s'adapte pas aux nouvelles fraudes)	Élevée (apprentissage continu)
Précision	Moyenne, dépend des règles des experts	Élevée, détecte des schémas complexes
Temps de traitement	Rapide mais limité	Plus lourd mais plus intelligent
Nouvelles fraudes	Difficile, mise à jour manuelle	Possible grâce à la détection d'anomalies

a. Méthodes traditionnelles

Les méthodes traditionnelles de détection des fraudes bancaires reposent principalement sur des règles définies par des experts et des seuils fixes. Bien que ces systèmes aient été relativement efficaces face aux schémas de fraude plus simples du passé, ils présentent aujourd'hui des limites critiques [13].

- **Systèmes basés sur des règles fixes (Rule-based)** : ces systèmes reposent sur des seuils et des conditions prédéfinis qui déclenchent des alertes lorsque certains critères sont remplis [8].
- **Analyse statistique simple** : cette approche utilise des indicateurs statistiques de base tels que la moyenne et l'écart-type pour identifier les transactions anormales [14].
- **Vérification manuelle** : les transactions suspectes sont examinées manuellement par des analystes, ce qui limite la capacité de traitement [8].

Ces méthodes présentent plusieurs limites importantes :

- Les règles statiques perdent de leur utilité à mesure que les techniques de fraude évoluent.
- Elles génèrent un nombre élevé de faux positifs, perturbant l'expérience client.
- Elles nécessitent une mise à jour manuelle constante, ce qui les rend coûteuses et peu adaptables [13].

b. Méthodes modernes (Machine Learning et Deep Learning)

Face aux limites des méthodes traditionnelles, les approches modernes basées sur le Machine Learning et le Deep Learning s'imposent comme des solutions efficaces pour la détection des fraudes bancaires. Contrairement aux systèmes basés sur des règles, ces méthodes s'adaptent et apprennent à partir de nouveaux schémas de fraude [13].

- **Apprentissage supervisé** : ces algorithmes sont entraînés à partir de jeux de données étiquetés contenant des transactions frauduleuses et légitimes. Les algorithmes les plus utilisés sont Random Forest, SVM et la Régression Logistique [13, 14].

Les figures 1.4 et 1.5 illustrent respectivement l'architecture du modèle Random Forest et le principe des SVM, deux algorithmes supervisés largement utilisés dans la détection des fraudes bancaires [17, 18].

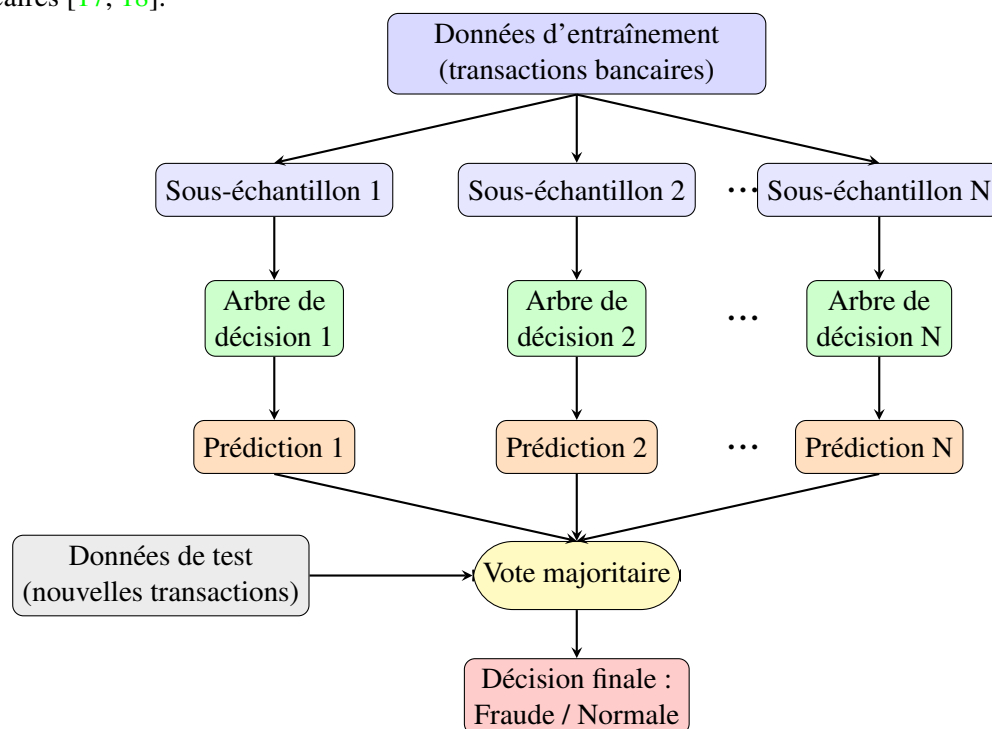


Figure 1.4 : Architecture du Random Forest

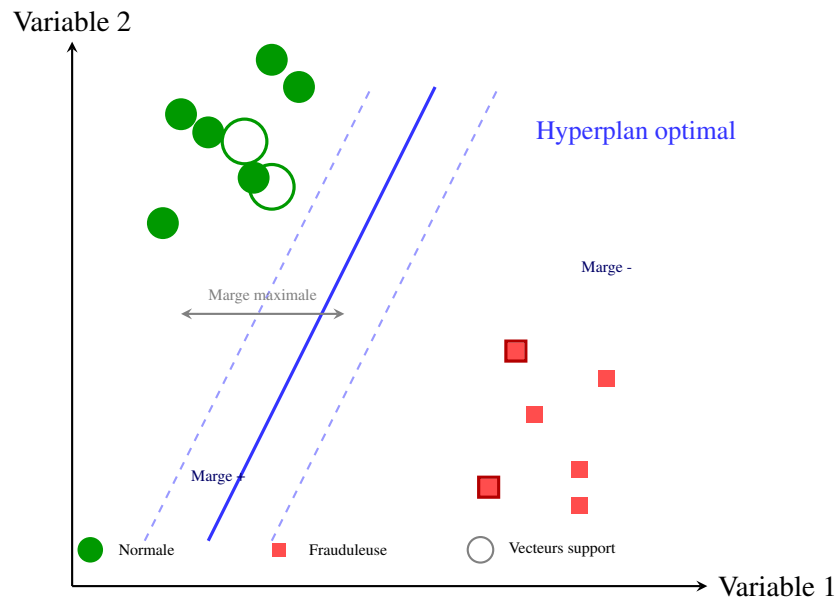


Figure 1.5 : Principe des SVM

- **Apprentissage non supervisé** : ces algorithmes identifient des anomalies ou des comportements suspects sans exemples préalablement étiquetés. L'algorithme K-Means est l'un des plus utilisés dans cette catégorie [13].

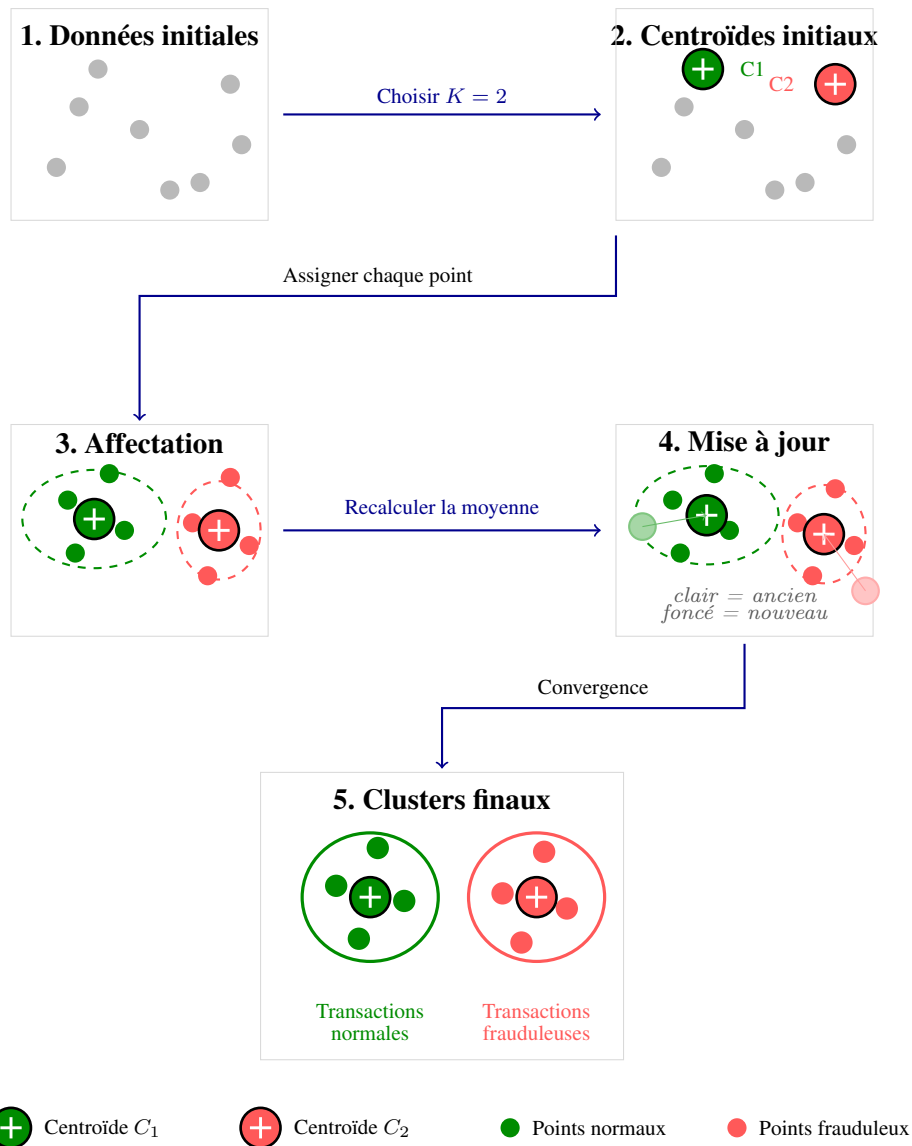


Figure 1.6 : Étapes de l'algorithme K-Means

- **Deep Learning** : les réseaux de neurones profonds sont capables de détecter des schémas complexes dans les données financières. Les Réseaux de Neurones Récurrents (RNN), les LSTM et les Autoencoders sont particulièrement efficaces pour l'analyse de séquences de transactions [7].

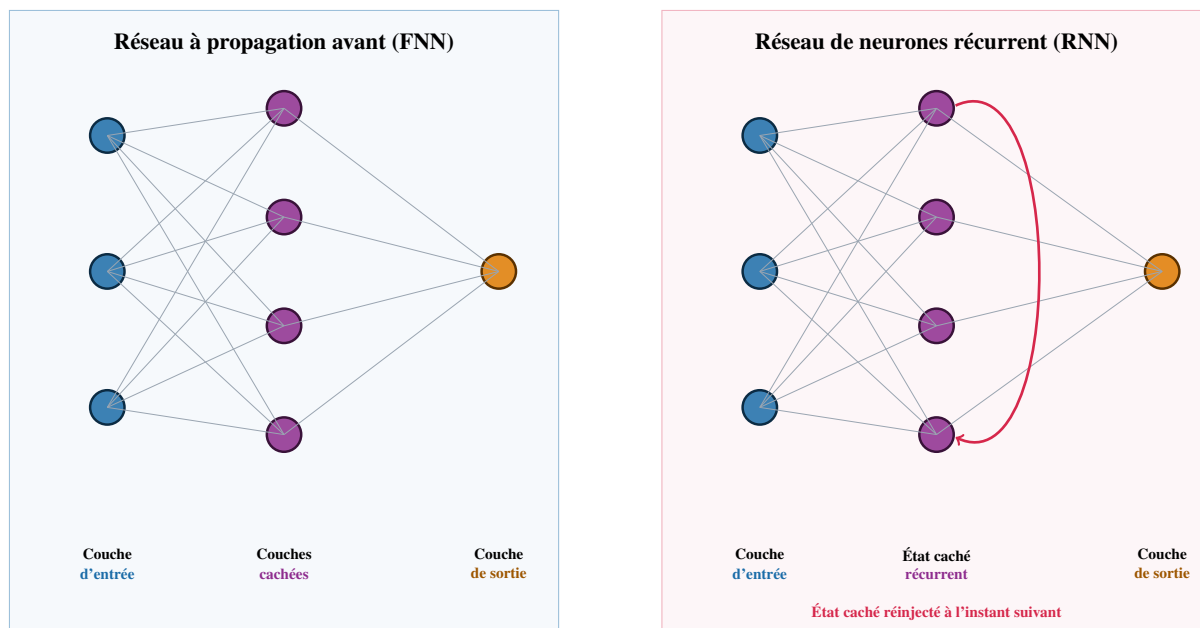


Figure 1.7 : Architecture du RNN comparée au FNN

- **Méthodes ensemblistes** : ces méthodes combinent plusieurs algorithmes pour améliorer la précision de détection [14].

Ces méthodes présentent plusieurs avantages :

- Elles s'adaptent automatiquement aux nouvelles techniques de fraude grâce à l'apprentissage continu.
- Elles réduisent considérablement le nombre de faux positifs.
- Elles peuvent analyser de grandes quantités de données en temps réel, 24 heures sur 24 [8].

1-6-3 Cadre réglementaire de la lutte contre la fraude bancaire

- **DSP2 / PSD2 (Directive sur les Services de Paiement 2)** : en 2007, la Commission européenne a promulgué la première directive sur les services de paiement pour deux raisons principales : créer un marché européen des paiements plus intégré, plus compétitif et plus efficace, et renforcer la sécurité des paiements à l'ère du numérique. La PSD2 est la deuxième version de cette directive ; bien qu'elle s'applique spécifiquement à l'Union européenne, elle est largement considérée comme une réglementation de référence, susceptible de modifier le mode de fonctionnement des banques, des processeurs de paiements et d'autres entreprises de services de paiements partout dans le monde [9]. Elle se concentre sur l'amélioration du marché des paiements de détail avec une sécurité renforcée et des offres plus compétitives, autour de quatre objectifs principaux [10] :
 - **Concurrence sur le marché** : en ouvrant l'accès aux données des clients, la DSP2 encourage de nouveaux acteurs, comme les entreprises du domaine de la fintech, à entrer sur le marché et à proposer des services financiers innovants, favorisant la concurrence et de meilleurs produits et services.

- **Sécurité** : la DSP2 impose des exigences de sécurité plus strictes pour les paiements en ligne, telles que l'authentification forte du client (SCA), permettant de réduire la fraude et de protéger les données financières des clients.
- **Protection des consommateurs** : la DSP2 donne aux clients plus de contrôle sur leurs données financières et renforce les procédures de traitement des réclamations, pour des résolutions plus rapides et plus efficaces.
- **Marché des paiements intégrés** : la DSP2 crée un marché européen des paiements plus intégré et plus efficace en uniformisant les réglementations en matière de paiement dans l'ensemble de l'UE, facilitant les paiements internationaux entre entreprises et clients.
- **RGPD** : le Règlement Général sur la Protection des Données (RGPD) de l'Union européenne (UE) est une législation sur la confidentialité des données, visant à donner aux individus un contrôle accru sur leurs informations personnelles. Cette loi introduit une exigence de juridiction extraterritoriale, ce qui signifie que les entreprises situées en dehors de l'UE qui collectent des données de résidents ou citoyens européens peuvent être tenues responsables [11].

Tableau 1.4 : Comparaison entre la DSP2/PSD2 et le RGPD.

Critère	DSP2 / PSD2	RGPD
Objectif principal	Sécuriser les paiements en ligne et ouvrir le marché aux fintechs	Protéger les données personnelles des individus
Champ d'application	Prestataires de services de paiement (banques, fintechs)	Toute entreprise traitant des données de résidents européens
Mécanisme clé	Authentification forte du client (SCA)	Minimisation des données, consentement, droit à l'explication
Portée géographique	Union européenne	Union européenne, avec effet extraterritorial
Lien avec la détection de fraude	Réduit la fraude à la source, avant la transaction	Encadre l'exploitation des données pour la détection a posteriori

1-7 Travaux connexes

Plusieurs travaux récents ont exploré l'utilisation d'algorithmes d'apprentissage automatique et profond pour la détection de fraudes et d'anomalies, avec des méthodologies proches de celle adoptée dans ce mémoire. **Hemanth Kumar et al. (2025)** [21] étudient l'application du Big Data et de l'IA à la détection de menaces de cybersécurité (malwares, ransomwares, intrusions) sur un jeu de données de 500 000 incidents. Leur étude compare quatre algorithmes — Random Forest, SVM, Deep Learning

(RNN) et K-Means — exactement les mêmes familles de méthodes que celles retenues dans ce mémoire. Leurs résultats montrent que le Deep Learning obtient la meilleure précision (96,8 suivi de Random Forest (94,1 atteint un taux de détection de 89,5 approches IA surpassent nettement les méthodes conventionnelles, tout en soulignant le coût computationnel du Deep Learning et sa vulnérabilité aux attaques adverses comme limites à résoudre. **Popova & Gardi (2025)** [22] se concentrent spécifiquement

sur la détection de fraude par carte bancaire, en comparant cinq modèles supervisés — Régression Logistique, Random Forest, XGBoost, KNN et MLP — sur le dataset ULB Credit Card Fraud Detection, le même que celui utilisé dans ce mémoire pour la configuration déséquilibrée. Leur contribution principale est méthodologique : ils insistent sur la nécessité d'évaluer les modèles sur l'ensemble de test **réel et non rééquilibré**, plutôt que sur un sous-ensemble artificiellement équilibré, afin de refléter fidèlement les conditions réelles de déploiement. Ils comparent également trois stratégies de rééquilibrage (sous-échantillonnage, SMOTE, et une approche hybride), et montrent que Random Forest reste le modèle le plus robuste, avec un F1-Score de 85,8 **Zhou, Zheng & Du (2025)** [23] proposent un

modèle hybride combinant RNN et LSTM pour la détection de fraude bancaire, également testé sur le dataset ULB. Leur approche associe l'architecture récurrente à une stratégie d'échantillonnage hybride afin de mieux capturer les dépendances temporelles entre transactions. Comparé à un LSTM seul, à KNN et à AdaBoost, leur modèle hybride RNN-LSTM améliore le F1-Score de 5,31 points (atteignant 85,44 Accuracy de 99,96 Ces trois travaux confirment la pertinence du choix des algorithmes retenus dans ce

mémoire (Random Forest, SVM, K-Means, RNN) pour la détection de fraude, et valident en particulier deux bonnes pratiques méthodologiques appliquées dans ce mémoire : l'évaluation sur l'ensemble de test réel non rééquilibré (comme le préconisent Popova & Gardi), et l'intérêt des architectures récurrentes pour capturer les dépendances temporelles entre transactions (comme le démontrent Zhou et al.). Ces éléments seront repris et discutés au Chapitre 3, où les résultats obtenus dans ce mémoire seront directement confrontés à ceux de Popova & Gardi et de Zhou et al.

1-8 Défis et limites des systèmes actuels de détection par IA

- **Faux positifs et surcharge d'informations** : l'un des défis majeurs reste encore aujourd'hui la gestion des faux positifs. Les systèmes d'IA peuvent parfois signaler des alertes erronées, et entraîner une surcharge d'informations à traiter pour les analystes en cybersécurité. Le risque : détourner l'attention des menaces réelles et ralentir la réaction face aux incidents critiques.
- **Utilisation de l'IA par les cybercriminels** : les cybercriminels exploitent également l'IA pour affiner leurs attaques. En utilisant des algorithmes sophistiqués, ils peuvent automatiser des attaques par phishing, détecter des vulnérabilités spécifiques, ou même contourner les systèmes de détection basés sur l'IA. Tout ceci rend la lutte contre les cybermenaces encore plus complexe.
- **Dépendance excessive aux systèmes automatisés** : autre enjeu en matière de cybersécurité, la dépendance accrue à l'automatisation via l'IA peut nuire à la vigilance humaine. Si les entreprises se fient uniquement à ces systèmes, elles risquent de négliger des aspects que l'IA ne peut pas toujours appréhender, comme l'intuition humaine ou la compréhension contextuelle d'une situation [12].

1-9 Conclusion

Ce chapitre a présenté les différents types de fraudes bancaires ainsi que les méthodes de détection existantes, des approches traditionnelles basées sur des règles aux techniques modernes d'apprentissage automatique, et les a situées par rapport à trois travaux connexes récents. Il ressort clairement que le Machine Learning et le Deep Learning constituent aujourd'hui les approches les plus adaptées pour faire face à la sophistication croissante des fraudes numériques et au volume massif des données transactionnelles. Le chapitre suivant présente l'approche proposée dans ce mémoire : l'architecture générale du système, la méthodologie détaillée de chaque algorithme et la démarche expérimentale adoptée.

Chapitre 2 : Approche Proposée

2-1 Introduction

Ce chapitre présente l'approche proposée dans ce mémoire pour la détection des fraudes bancaires par analyse massive de données et intelligence artificielle. Après avoir établi dans le chapitre précédent les bases théoriques sur la fraude bancaire et les méthodes de détection existantes, la méthodologie concrète mise en œuvre pour construire un système de détection performant est décrite ici. L'objectif principal

de ce chapitre est d'expliquer la démarche suivie, depuis le choix des données jusqu'à la classification finale des transactions. Pour cela, deux datasets réels de transactions bancaires sont exploités et quatre algorithmes d'apprentissage automatique sont appliqués : Random Forest, SVM, K-Means et RNN, comparés sur les deux configurations de données afin d'identifier le modèle le plus adapté à la détection des fraudes bancaires. Ce chapitre s'articule autour de quatre axes principaux : l'architecture générale du

système proposé, la présentation détaillée de chaque algorithme utilisé, l'extraction des caractéristiques à partir des données transactionnelles, et enfin la phase de classification et d'évaluation des modèles.

2-2 Architecture générale du système proposé

2-2-1 Démarche adoptée

La démarche adoptée dans ce chapitre s'articule autour d'un pipeline structuré en plusieurs étapes complémentaires, détaillées dans les sections suivantes. La section 2-2-2 présente le **Pipeline du système proposé**, illustrant l'enchaînement global des étapes depuis le chargement des données jusqu'à la décision finale de classification. La section 2-3 détaille la **Méthodologie** suivie pour chacun des quatre algorithmes retenus : Random Forest, SVM, K-Means et RNN (avec sa variante LSTM). Le **Prétraitement et normalisation** des données, propres à chaque dataset et à chaque algorithme, sont présentés en section 2-4. L'**Extraction des caractéristiques**, incluant la sélection des variables d'entrée, la gestion du déséquilibre des classes (SMOTE, pondération, sous-échantillonnage) ainsi que la réduction de dimension, fait l'objet de la section 2-5. La **Classification**, reposant sur l'entraînement des modèles et l'application de règles de décision adaptées à chaque algorithme, est décrite en section 2-6. Enfin, les performances de chaque modèle sont évaluées et comparées à l'aide de cinq métriques : Accuracy, Précision, Recall, F1-Score et False Positive Rate.

2-2-2 Pipeline du système proposé

La figure 2.1 illustre le pipeline global du système de détection proposé. Ce pipeline représente l'enchaînement logique des étapes suivies, depuis la collecte des données brutes jusqu'à la décision finale de classification d'une transaction en normale ou frauduleuse. Le prétraitement effectif diffère

toutefois d'un algorithme à l'autre, en particulier sur le dataset déséquilibré. Les paragraphes suivants détaillent, étape par étape, le pipeline exact suivi pour chaque algorithme et chaque dataset.

Étapes communes à tous les algorithmes

Quel que soit l'algorithme ou le dataset considéré, le pipeline débute systématiquement par les étapes suivantes :

1. Chargement du fichier CSV (`creditcard.csv` ou `creditcard2023.csv`) à l'aide de `pandas.read_csv`.
2. Affichage de la forme du dataset (`shape`) et de la distribution de la variable cible `Class` (`value_counts`).
3. Vérification des valeurs manquantes (`isnull().sum()`) – aucune détectée sur les deux datasets.
4. Suppression de la colonne `id` (dataset `creditcard2023` uniquement, sans effet sur `creditcard.csv` qui ne la possède pas).
5. Séparation des variables explicatives `X` (toutes les colonnes sauf `Class`) et de la variable cible `y` (`Class`).

Pipeline sur le dataset équilibré (`creditcard2023`)

Sur ce dataset, les quatre algorithmes (Random Forest, SVM, RNN, K-Means) suivent exactement le même pipeline de prétraitement :

1. Un unique `StandardScaler` est ajusté (`fit_transform`) sur l'ensemble des 29 variables explicatives (`V1-V28` et `Amount`) en une seule opération.
2. Division du jeu de données normalisé en ensembles d'entraînement et de test (80 %/20 %), stratifiée sur `Class` (`train_test_split`, `random_state=42`), le dataset étant naturellement équilibré (50 %/50 %), aucun rééchantillonnage n'est nécessaire.
3. **RNN uniquement** : les données normalisées sont ensuite transformées au format séquentiel attendu par la couche récurrente, via un *reshape* de forme (échantillons, 1, features) (un seul pas de temps par transaction).
4. **K-Means uniquement** : un sous-ensemble de validation supplémentaire (10 % du train) est extrait avant l'application de la méthode du coude ($K = 2$ à 8) et l'entraînement final avec $K = 2$.

Pipeline sur le dataset déséquilibré (`creditcard`) – Random Forest

Random Forest est le seul algorithme entraîné sur l'ensemble réel complet, sans rééchantillonnage :

1. Un unique `StandardScaler` est ajusté sur l'ensemble des 30 variables explicatives (`Time`, `V1-V28`, `Amount`) en une seule opération.

2. Division en ensembles d'entraînement (227 845 transactions) et de test (56 962 transactions), stratifiée sur `Class` (80 %/20 %).
3. Entraînement direct du modèle sur l'ensemble d'entraînement réel, le déséquilibre étant compensé uniquement par le paramètre `class_weight='balanced'` du classifieur.
4. Test de sept seuils de probabilité (0,1 à 0,7) sur l'ensemble de validation croisée pour sélectionner le seuil maximisant le F1-Score avant l'évaluation finale sur le test.

Pipeline sur le dataset déséquilibré (creditcard) – SVM

Le pipeline du SVM comporte davantage d'étapes, en particulier au niveau du rééquilibrage :

1. Les variables `Amount` et `Time` sont normalisées **séparément**, chacune à l'aide de son propre `StandardScaler`, produisant `Amount_scaled` et `Time_scaled`. Les colonnes originales `Amount` et `Time` sont ensuite supprimées.
2. Division en ensembles d'entraînement et de test (80 %/20 %), stratifiée sur `Class`.
3. Sur l'ensemble d'entraînement uniquement : sous-échantillonnage aléatoire de la classe majoritaire, réduisant les transactions normales à 20 000 exemples (contre 227 451 initialement), conservées avec la totalité des 394 fraudes d'entraînement.
4. Application de **SMOTE** sur ce sous-ensemble réduit, avec `sampling_strategy=0.1`, générant des exemples synthétiques de fraude jusqu'à atteindre un ratio d'environ 10 % de fraudes.
5. Un **nouveau** `StandardScaler` est ajusté sur cet ensemble d'entraînement final (rééquilibré), puis utilisé pour transformer l'ensemble de test – les variables `Amount_scaled` et `Time_scaled` subissent ainsi une double normalisation successive.
6. Entraînement du SVM (kernel RBF) sur l'ensemble d'entraînement final.
7. Test de sept seuils de probabilité (0,1 à 0,7) pour sélectionner le seuil optimal (0,4) maximisant le F1-Score.

Pipeline sur le dataset déséquilibré (creditcard) – RNN et LSTM

RNN et LSTM suivent un pipeline identique, à l'exception de la taille du sous-ensemble équilibré et de l'architecture du réseau :

1. Les variables `Amount` et `Time` sont normalisées séparément (deux `StandardScaler` distincts), puis les colonnes originales sont supprimées, comme pour le SVM.
2. Un sous-ensemble équilibré est constitué par **tirage aléatoire simple** (`sample`, sans SMOTE) : les 492 fraudes sont conservées intégralement, complétées par un échantillon aléatoire de transactions normales – **492** pour le RNN (984 échantillons au total), **2 000** pour le LSTM (2 492 échantillons au total).
3. Division de ce sous-ensemble en train/test (80 %/20 %), stratifiée sur `Class`.

4. Un `StandardScaler` unique est ajusté sur l'ensemble d'entraînement du sous-ensemble équilibré, puis appliqué au test.
5. Les données normalisées sont transformées au format séquentiel (échantillons, 1, features) pour l'entrée du réseau récurrent.
6. **RNN** : architecture `SimpleRNN(64) → Dropout(0.3) → SimpleRNN(32) → Dropout(0.3) → Dense(16, relu) → Dense(1, sigmoid)`; optimiseur Adam (`lr=0.001`), `EarlyStopping` (`patience=10`), jusqu'à 100 époques, `batch_size=32`, 20 % du train réservés à la validation interne.
7. **LSTM** : architecture `LSTM(128) → Dropout(0.3) → LSTM(64) → Dropout(0.3) → LSTM(32) → Dropout(0.2) → BatchNormalization → Dense(16, relu) → Dense(1, sigmoid)`; optimiseur Adam (`lr=0.0005`), `EarlyStopping` (`patience=15`) combiné à `ReduceLROnPlateau` (`factor=0.5`, `patience=5`, `min_lr=0.00001`), jusqu'à 150 époques, `batch_size=16`.
8. Classification finale au seuil par défaut de 0,5 sur les probabilités prédites par le réseau.

Pipeline sur le dataset déséquilibré (creditcard) – K-Means et K-Means+PCA

K-Means et sa variante avec PCA partagent le même sous-ensemble équilibré et le même schéma de division des données, mais diffèrent sur la réduction de dimension et certains hyperparamètres :

1. Les variables `Amount` et `Time` sont normalisées séparément (deux `StandardScaler` distincts), puis les colonnes originales sont supprimées.
2. Un sous-ensemble équilibré est constitué par tirage aléatoire des 492 fraudes et de 492 transactions normales (984 échantillons au total), sans SMOTE.
3. Division en train/validation/test selon un schéma **60 %/20 %/20 %**, stratifiée sur `Class` (`train_test_split` appliqué en deux temps).
4. Un `StandardScaler` unique est ajusté sur le train, puis appliqué à la validation et au test.
5. **K-Means+PCA uniquement** : une réduction de dimension par PCA (`n_components=15`) est appliquée sur les données normalisées, conservant 93,1 % de la variance expliquée ; le PCA est ajusté sur le train puis appliqué à la validation et au test.
6. Méthode du coude : calcul de l'inertie pour $K = 2$ à 8, confirmant le choix de $K = 2$ (cohérent avec la nature binaire du problème).
7. Entraînement de `KMeans` avec $K = 2$: `n_init=10` pour K-Means simple, `init='k-means++'`, `n_init=50`, `max_iter=500` pour la variante K-Means+PCA (paramètres plus poussés du fait de la recherche de convergence sur l'espace réduit).
8. Identification du cluster correspondant à la fraude (celui dont le taux de fraude moyen est le plus élevé), puis classification finale de chaque transaction selon son cluster d'appartenance.

9. Calcul du Silhouette Score sur chacun des trois ensembles (train, validation, test) pour évaluer la qualité de la séparation des clusters.

Architecture du système proposé

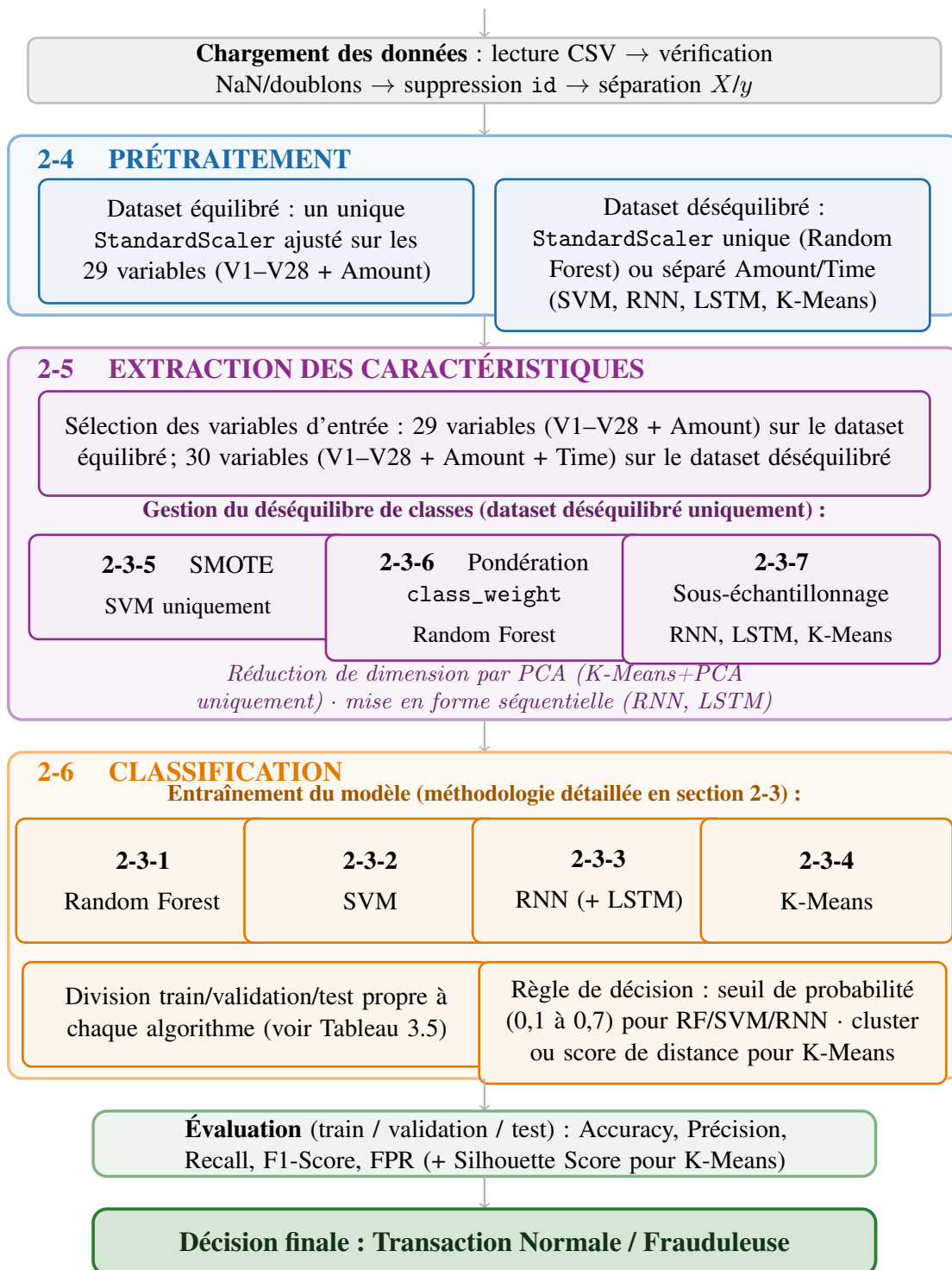


Figure 2.1 : Pipeline global du système de détection proposé

2-3 Méthodologie

Cette section présente les quatre algorithmes d'apprentissage automatique utilisés dans ce mémoire, en détaillant pour chacun sa définition théorique, son principe de fonctionnement, ainsi que la manière dont il a été configuré et appliqué sur les deux datasets.

2-3-1 Random Forest

Random Forest est un algorithme d'apprentissage supervisé qui fonctionne comme un ensemble de multiples arbres de décision. Chaque arbre est entraîné sur un sous-échantillon des données obtenu par tirage aléatoire avec remise (bootstrap), et sur un sous-ensemble aléatoire des variables (features). La prédiction finale du modèle résulte d'un vote majoritaire entre tous les arbres pour la classification [17]. Cette approche d'agrégation permet de réduire le surapprentissage (overfitting) et d'améliorer la robustesse du modèle par rapport à un arbre de décision unique. L'algorithme 1 résume les principales étapes de fonctionnement du Random Forest tel qu'implémenté dans ce mémoire.

Algorithme 1 : Random Forest

1. Charger le dataset et le prétraiter
2. Diviser les données en ensembles d'entraînement et de test
3. Initialiser un nombre N d'arbres de décision
4. Pour chaque arbre de décision :
 - a. Échantillonner aléatoirement les données avec remise (bootstrap)
 - b. Entraîner l'arbre sur un sous-ensemble aléatoire de features
5. Agréger les prédictions de tous les arbres
6. Utiliser le vote majoritaire pour la classification finale
7. Évaluer la performance du modèle sur les données de test

Dans ce mémoire, le Random Forest a été configuré avec les paramètres suivants :

Tableau 2.1 : Paramètres du modèle Random Forest

Paramètre	Valeur
Nombre d'arbres (n_estimators)	100
Profondeur maximale (max_depth)	Non limitée
Échantillons min par feuille (min_samples_leaf)	1
Échantillons min par division (min_samples_split)	2
Critère de division (max_features)	sqrt
Bootstrap	True
Pondération des classes (class_weight)	balanced
Graine aléatoire (random_state)	42
Parallélisation (n_jobs)	-1 (tous les cœurs)

La robustesse du modèle a été évaluée par validation croisée à 5 plis (cross_val_score, cv=5), avec le F1-Score comme métrique de scoring (scoring='f1'), en complément de l'évaluation classique sur l'ensemble de test.

Random Forest a été appliqué sur les deux datasets : le dataset déséquilibré (creditcard) et le dataset équilibré (creditcard2023), ce qui a permis de comparer directement sa robustesse face au déséquilibre de classes. Pour la classification finale, plusieurs seuils de probabilité (0,1 à 0,7) ont été testés afin d'identifier le seuil optimal maximisant le F1-Score.

2-3-2 Support Vector Machine (SVM)

Le Support Vector Machine (SVM) est un algorithme d'apprentissage supervisé qui classe les données en trouvant un hyperplan optimal maximisant la marge entre les deux classes. Pour les données non linéairement séparables, SVM utilise une fonction noyau (kernel) qui projette les données dans un espace de dimension supérieure où une séparation linéaire devient possible [18]. L'algorithme 2 résume les principales étapes de fonctionnement du SVM tel qu'implémenté dans ce mémoire.

Algorithme 2 : Support Vector Machine

1. Charger le dataset et le prétraiter
2. Diviser les données en ensembles d'entraînement et de test
3. Normaliser les données avec StandardScaler
4. Sélectionner la fonction noyau (linéaire ou RBF)
5. Trouver l'hyperplan optimal qui maximise la marge
6. Entraîner le modèle SVM sur les données d'entraînement
7. Prédire les classes pour les données de test
8. Évaluer Accuracy, Précision, Recall, F1 et FPR

Dans ce mémoire, deux configurations de SVM ont été utilisées selon le dataset : sur le dataset équilibré (creditcard2023), un kernel linéaire (LinearSVC) a été retenu, plus adapté aux grands volumes de données et aux features déjà transformées par PCA ; sur le dataset déséquilibré (creditcard), un kernel RBF combiné avec SMOTE a été utilisé, afin de gérer le déséquilibre de classes et de capturer les relations non linéaires entre les features.

Tableau 2.2 : Paramètres du modèle SVM

Paramètre	Dataset équilibré	Dataset déséquilibré
Kernel	Linéaire (LinearSVC)	RBF
C (régularisation)	1.0	1.0
Gamma	– (non applicable)	scale (default)
Pondération des classes (class_weight)	–	balanced
Estimation de probabilité	–	probability=True
Normalisation	StandardScaler	StandardScaler (x2)
Rééquilibrage	Non nécessaire	Undersampling + SMOTE + class_weight
Graine aléatoire (random_state)	42	42
Seuil optimal	0.4	0.4

Sur le dataset déséquilibré, la gestion du déséquilibre repose ainsi sur **trois mécanismes cumulés** : le sous-échantillonnage de la classe majoritaire, l’application de SMOTE, et la pondération des classes (class_weight='balanced') directement au niveau du classifieur SVM. La validation croisée associée s’appuie sur 3 plis (cv=3), avec un calcul séparé de l’Accuracy, de la Précision, du Recall et du F1-Score pour chaque pli.

2-3-3 Réseaux de Neurones Récurrents (RNN)

Les réseaux de neurones récurrents (RNN) sont une famille de modèles d'apprentissage profond conçus pour traiter des données séquentielles. Contrairement aux réseaux de neurones classiques, un RNN possède une mémoire interne (état caché) qui lui permet de conserver une information sur les entrées précédentes et de l'utiliser pour influencer le traitement de l'entrée courante. L'entraînement s'appuie sur l'algorithme de rétropropagation à travers le temps (*Backpropagation Through Time*, BPTT), qui propage l'erreur à travers les différents pas de temps du réseau [20].

L'algorithme 3 résume les principales étapes de fonctionnement du RNN tel qu'implémenté dans ce mémoire.

Algorithme 3 : RNN (Recurrent Neural Network)

1. Charger le dataset et le prétraiter (normalisation StandardScaler)
2. Restructurer les données au format séquentiel (samples, timesteps, features)
3. Diviser les données en ensembles d'entraînement et de test
4. Définir l'architecture du réseau (couches SimpleRNN + Dropout + Dense)
5. Initialiser les poids (glorot_uniform) et les biais (zéro)
6. Entraîner le modèle par rétropropagation à travers le temps (BPTT)
7. Mettre à jour les poids par descente de gradient (optimiseur Adam)
8. Tester le modèle sur les données non vues
9. Optimiser le seuil de classification (test de 0,1 à 0,7)
10. Évaluer Accuracy, Précision, Recall, F1 et FPR

Le RNN a été appliqué sur les deux datasets, mais avec des configurations différentes en raison de leur taille et de leur distribution.

Sur le dataset équilibré (creditcard2023), le RNN a été entraîné directement sur l'ensemble complet (568 630 transactions), sans nécessité de rééchantillonnage. L'architecture retenue est la suivante : SimpleRNN(128) → Dropout(0,3) → BatchNormalization → SimpleRNN(64) → Dropout(0,3) → BatchNormalization → Dense(32, relu) → Dropout(0,2) → Dense(1, sigmoid). L'entraînement utilise l'optimiseur Adam (lr=0,001), jusqu'à 50 époques, avec un batch_size de 512 et 10 % du train réservés à la validation interne. Les callbacks EarlyStopping (patience=5) et ReduceLROnPlateau (factor=0,5, patience=3, min_lr=0,00001) sont utilisés conjointement.

Sur le dataset déséquilibré (creditcard), le RNN a été entraîné sur un sous-ensemble équilibré (492 fraudes + 492 transactions normales échantillonnées aléatoirement), afin de compenser la rareté des fraudes durant l'apprentissage. L'architecture retenue est plus légère, adaptée à ce sous-ensemble de taille réduite : SimpleRNN(64) → Dropout(0,3) → SimpleRNN(32) → Dropout(0,3) → Dense(16, relu) → Dense(1, sigmoid). L'entraînement utilise l'optimiseur Adam (lr=0,001), jusqu'à 100 époques, avec un batch_size de 32 et 20 % du train réservés à la validation interne. Seul le callback EarlyStopping (patience=10) est utilisé ici (sans ReduceLROnPlateau).

Sur ce même dataset déséquilibré, une variante **LSTM** (*Long Short-Term Memory*) a également été testée, une architecture dotée de portes (*gates*) permettant de mieux réguler le flux d'information et de limiter le problème de disparition du gradient. Son architecture est plus profonde que celle du RNN : LSTM(128) → Dropout(0,3) → LSTM(64) → Dropout(0,3) → LSTM(32) → Dropout(0,2) → BatchNormalization → Dense(16, relu) → Dense(1, sigmoid), entraînée sur l'optimiseur Adam (lr=0,0005), jusqu'à 150 époques, avec un batch_size de 16, combinant les callbacks EarlyStopping (patience=15) et ReduceLROnPlateau (factor=0,5, patience=5, min_lr=0,00001).

Dans les trois configurations, les poids sont initialisés selon le schéma glorot_uniform (par défaut Keras), les biais à zéro, et les poids récurrents selon un schéma orthogonal (par défaut pour SimpleRNN). L'entraînement s'appuie sur la rétropropagation à travers le temps (BPTT) et la fonction de perte *Binary Crossentropy*.

Tableau 2.3 : Paramètres des modèles RNN et LSTM

Paramètre	RNN (équilibré)	RNN (déséquilibré)	LSTM (déséquilibré)
Architecture	SimpleRNN(128,64)	SimpleRNN(64,32)	LSTM(128,64,32)
Optimiseur	Adam (lr = 0,001)	Adam (lr = 0,001)	Adam (lr = 0,0005)
Époques (max)	50	100	150
Batch size	512	32	16
Callbacks	EarlyStopping(5) + ReduceLROnPlateau	EarlyStopping(10) seul	EarlyStopping(15) + ReduceLROnPlateau

2-3-4 K-Means

K-Means est un algorithme d'apprentissage non supervisé qui partitionne un ensemble de données en K groupes (clusters) homogènes, en se basant sur la similarité entre les observations. Le principe consiste à initialiser K centroïdes, puis à assigner itérativement chaque point de données au centroïde le plus proche (distance euclidienne), avant de recalculer la position des centroïdes comme la moyenne des points qui leur sont assignés [19].

L'algorithme 4 résume les principales étapes de fonctionnement du K-Means tel qu'implémenté dans ce mémoire.

Algorithme 4 : K-Means

1. Charger le dataset et le prétraiter (normalisation StandardScaler)
2. Choisir le nombre de clusters K (méthode du coude)
3. Initialiser K centroïdes aléatoirement (k-means++)
4. Assigner chaque point de données au centroïde le plus proche
5. Recalculer la position de chaque centroïde
6. Répéter les étapes 4 et 5 jusqu'à convergence
7. Identifier le cluster correspondant à la fraude
8. Évaluer Accuracy, Précision, Recall, F1, FPR et Silhouette Score

Le choix de $K = 2$ a été confirmé par la méthode du coude (test de $K = 2$ à 8), en cohérence avec la nature binaire du problème.

Tableau 2.4 : Paramètres du modèle K-Means

Paramètre	K-Means (simple)	K-Means + PCA
Nombre de clusters (K)	2	2
Méthode d'initialisation	k-means++ (default)	k-means++ (explicite)
Nombre d'initialisations (n_init)	10	50
Itérations maximales (max_iter)	300 (default)	500
Distance	Euclidienne	Euclidienne

K-Means a été appliqué sur les deux datasets. Sur le dataset équilibré, l'algorithme a été entraîné directement sur l'ensemble des transactions, avec les paramètres par défaut de scikit-learn ($n_init=10$, $max_iter=300$); en raison du coût de calcul du Silhouette Score sur un grand nombre d'observations, celui-ci est estimé sur un échantillon aléatoire de 5 000 points du test plutôt que sur l'ensemble complet. Sur le dataset déséquilibré, K-Means a été appliqué sur un sous-ensemble équilibré (492 fraudes + 492 transactions normales), avec également une variante testée après réduction de dimension par PCA (15 composantes, 93,1 % de variance expliquée). Pour cette variante K-Means+PCA, des paramètres plus poussés ont été utilisés ($n_init=50$, $max_iter=500$) afin de favoriser la convergence de l'algorithme sur l'espace de dimension réduite.

2-3-5 SMOTE (Synthetic Minority Over-sampling Technique)

SMOTE (*Synthetic Minority Over-sampling Technique*) est une technique de rééchantillonnage qui génère des exemples synthétiques de la classe minoritaire (la fraude), plutôt que de simplement dupliquer les observations existantes. Pour chaque exemple minoritaire, SMOTE sélectionne l'un de ses plus proches voisins (dans le même espace de caractéristiques) et crée un nouvel exemple synthétique par interpolation linéaire entre les deux points, ce qui permet d'enrichir la classe minoritaire sans surapprentissage direct sur des copies identiques.

Dans ce mémoire, **SMOTE n'est utilisé que pour le SVM**, et uniquement sur le dataset déséquilibré (creditcard). Il est appliqué après un sous-échantillonnage préalable de la classe majoritaire (réduite à 20 000 transactions normales, contre 227 451 initialement), avec le paramètre

sampling_strategy=0.1 : SMOTE génère alors des exemples synthétiques de fraude jusqu'à atteindre un ratio d'environ 10 % de fraudes dans l'ensemble d'entraînement final, et non un rééquilibrage complet à 50/50. Ce choix d'un ratio partiel, plutôt qu'un rééquilibrage total, vise à éviter la génération d'un nombre excessif d'exemples synthétiques potentiellement peu représentatifs de la réalité des fraudes.

Aucun autre algorithme n'utilise SMOTE dans ce mémoire :

- **Random Forest** ne retire ni ne synthétise aucune donnée, s'appuyant uniquement sur la pondération des classes (voir section 2-3-6).
- **RNN et LSTM** utilisent un sous-ensemble équilibré constitué par simple tirage aléatoire (492 fraudes + 492 ou 2 000 transactions normales), sans génération d'exemples synthétiques.
- **K-Means** (et sa variante avec PCA) utilise le même principe de tirage aléatoire équilibré, la nature non supervisée de l'algorithme rendant SMOTE non pertinent pour cette approche.

Sur le dataset équilibré (creditcard2023), naturellement réparti à 50 %/50 % entre les deux classes, SMOTE n'est appliqué pour aucun des quatre algorithmes, un rééchantillonnage n'étant pas nécessaire.

2-3-6 Pondération des classes (class_weight='balanced')

La pondération des classes (class_weight='balanced') est un mécanisme intégré directement au niveau du classifieur, qui ajuste automatiquement le poids attribué à chaque classe dans le calcul de la fonction de perte, en fonction de leur fréquence inverse dans les données d'entraînement. Contrairement à SMOTE ou au sous-échantillonnage, cette approche ne modifie ni la taille ni la composition du jeu de données : elle agit uniquement sur la façon dont les erreurs de classification sont pénalisées durant l'apprentissage, en pénalisant davantage les erreurs commises sur la classe minoritaire (la fraude).

Dans ce mémoire, class_weight='balanced' est utilisé par **deux algorithmes** :

- **Random Forest** : c'est l'**unique** mécanisme de gestion du déséquilibre. Aucune donnée n'est retirée ni synthétisée ; le modèle est entraîné directement sur l'ensemble réel complet (227 845 transactions d'entraînement), et le déséquilibre extrême (0,17 % de fraudes) est compensé uniquement par cette pondération au niveau du classifieur.
- **SVM** : class_weight='balanced' est utilisé en **complément** du sous-échantillonnage et de SMOTE (voir section 2-3-5), ajoutant une pondération supplémentaire au niveau de la fonction de perte du classifieur, en plus du rééquilibrage déjà effectué sur les données elles-mêmes. La gestion du déséquilibre pour le SVM repose ainsi sur **trois mécanismes cumulés**.

RNN, LSTM et K-Means n'utilisent pas class_weight, ces algorithmes reposant exclusivement sur la construction d'un sous-ensemble équilibré par tirage aléatoire en amont de l'entraînement (voir sections 2-3-3 et 2-3-4).

Sur le dataset équilibré (creditcard2023), naturellement réparti à 50 %/50 %, l'usage de class_weight='balanced' n'est pas nécessaire et n'a donc été appliqué à aucun des quatre algorithmes.

2-3-7 Sous-échantillonnage aléatoire (undersampling)

Le sous-échantillonnage aléatoire (*random undersampling*) consiste à réduire la classe majoritaire (les transactions normales) par tirage aléatoire simple, sans génération de nouvelles données, afin d'obtenir un jeu de données de taille réduite mais équilibré entre les deux classes. Contrairement à SMOTE, cette technique n'introduit aucune observation synthétique : elle se contente de sélectionner un sous-ensemble représentatif de la classe majoritaire.

Dans ce mémoire, le sous-échantillonnage aléatoire est utilisé par **trois algorithmes**, tous sur le dataset déséquilibré, comme méthode **unique et suffisante** de gestion du déséquilibre (sans combinaison avec SMOTE ni `class_weight`) :

- **RNN** : l'ensemble des 492 fraudes est conservé, complété par un tirage aléatoire de 492 transactions normales, formant un sous-ensemble équilibré de 984 transactions au total.
- **LSTM** : même principe, mais avec un tirage aléatoire de 2 000 transactions normales (au lieu de 492), afin de disposer d'un jeu d'entraînement de taille plus importante pour ce modèle plus profond, soit 2 492 transactions au total.
- **K-Means (et sa variante avec PCA)** : l'ensemble des 492 fraudes est conservé, complété par un tirage aléatoire de 492 transactions normales, formant également un sous-ensemble équilibré de 984 transactions. Ce choix se justifie par la nature non supervisée de l'algorithme, pour lequel un sous-ensemble strictement équilibré facilite l'identification des deux clusters (normal et fraude) sans qu'un rééquilibrage plus sophistiqué comme SMOTE ne soit nécessaire.

Contrairement au Random Forest (pondération seule, sur données réelles complètes) et au SVM (undersampling combiné à SMOTE et à la pondération), ces trois algorithmes reposent donc sur le sous-échantillonnage aléatoire comme **seule et unique** stratégie de gestion du déséquilibre, ce qui explique la taille nettement plus réduite (984 ou 2 492 transactions) de leurs ensembles d'entraînement et de test par rapport au Random Forest (284 807 transactions).

2-4 Prétraitement et normalisation

Avant l'entraînement des modèles, plusieurs opérations de prétraitement ont été appliquées sur les variables d'entrée : vérification des valeurs manquantes (aucune détectée sur les deux datasets), suppression de la colonne `id` (dataset `creditcard2023` uniquement), et normalisation des variables. Contrairement à une démarche uniforme, la normalisation effective diffère selon l'algorithme et le dataset considérés.

Sur le dataset équilibré (`creditcard2023`), les quatre algorithmes (Random Forest, SVM, RNN, K-Means) appliquent la même procédure : un unique `StandardScaler` est ajusté (`fit_transform`) sur l'ensemble des variables explicatives (`V1–V28` et `Amount`) en une seule opération, avant la division entraînement/test.

Sur le dataset déséquilibré (`creditcard`), le prétraitement diverge selon l'algorithme :

- **Random Forest** : un unique `StandardScaler` est ajusté sur l'ensemble des 30 variables (`Time`, `V1–V28`, `Amount`) en une seule opération, avant la division entraînement/test. Random Forest est le seul algorithme entraîné directement sur l'ensemble réel complet (227 845 transactions d'entraînement, 56 962 de test), sans rééchantillonnage ni réduction de la taille des données.
- **SVM, RNN, LSTM et K-Means** : les variables `Amount` et `Time` sont normalisées **séparément**, à l'aide de deux instances distinctes de `StandardScaler` (une pour chaque variable), donnant `Amount_scaled` et `Time_scaled`. Les colonnes originales `Amount` et `Time` sont ensuite supprimées du jeu de données. Les composantes PCA (`V1–V28`), déjà centrées-réduites par la transformation d'origine, ne sont pas re-normalisées à cette étape.

Pour le SVM spécifiquement, une **seconde** normalisation est appliquée : un nouveau `StandardScaler` est ajusté sur l'ensemble d'entraînement final, après sous-échantillonnage et SMOTE (voir section 4.3), puis utilisé pour transformer l'ensemble de test. Les variables `Amount_scaled` et `Time_scaled` subissent ainsi une double normalisation successive.

2-5 Extraction des caractéristiques

Les deux datasets utilisés dans ce mémoire partagent une structure commune de 28 variables anonymisées (`V1` à `V28`), issues d'une transformation par Analyse en Composantes Principales (PCA). Cette transformation, appliquée en amont par les fournisseurs des datasets pour des raisons de confidentialité, projette les variables originales (identité du porteur de carte, localisation, type de commerçant, etc.) dans un nouvel espace de représentation où chaque composante V_i capture une part de la variance des données originales, sans révéler d'information directement identifiable.

En complément de ces 28 composantes, chaque transaction est caractérisée par : **Amount**, le montant de la transaction, présent dans les deux datasets ; **Time**, le temps écoulé (en secondes) depuis la première transaction, disponible uniquement dans le dataset déséquilibré ; et **Class**, la variable cible (0 = normale, 1 = fraude). La colonne `id`, présente uniquement dans le dataset `creditcard2023` et ne portant

aucune information prédictive, est systématiquement supprimée lors du chargement des données, pour l'ensemble des algorithmes et des deux datasets.

Le nombre exact de variables d'entrée (features) fournies à chaque modèle varie toutefois selon le prétraitement appliqué. Sur le dataset équilibré, les quatre algorithmes reçoivent 29 variables en entrée (V1 à V28 + Amount). Sur le dataset déséquilibré, Random Forest reçoit 30 variables (Time, V1 à V28, Amount) telles quelles; les autres algorithmes (SVM, RNN, LSTM, K-Means) reçoivent également 30 variables, mais où Amount et Time originaux sont remplacés par leurs versions normalisées Amount_scaled et Time_scaled (voir section 4.2), les 28 composantes V1–V28 restant identiques. Pour la variante K-Means+PCA, ces 30 variables sont ensuite réduites à 15 composantes principales par l'application d'une seconde PCA (voir section 4.3).

Tableau 2.5 : Structure comparée des deux datasets

Caractéristique	Dataset déséquilibré	Dataset équilibré
Nombre de transactions	284 807	568 630
Nombre de variables	31 (Time, V1–V28, Amount, Class)	31 (id, V1–V28, Amount, Class)
Variables PCA	V1 à V28	V1 à V28
Variable temporelle	Time (secondes)	Absente
Montant moyen (Amount)	≈ 88,35 €	≈ 12 041,96 €

2-6 Classification

La phase de classification constitue l'étape finale du pipeline, où chaque transaction est étiquetée comme normale ou frauduleuse à partir des modèles entraînés. Cette section décrit la division des données – qui varie sensiblement d'un algorithme à l'autre selon la taille de l'ensemble effectivement utilisé (section 5.1) –, la règle de décision et le seuil de classification propres à chaque algorithme (section 5.2), les métriques d'évaluation retenues (section 5.3), ainsi que la démarche de comparaison des quatre algorithmes étudiés (section 5.4).

2-6-1 Division des données

Contrairement à un schéma unique appliqué uniformément, la division des données varie selon l'algorithme et le dataset considérés, en fonction de la taille de l'ensemble effectivement utilisé pour l'entraînement.

Sur le dataset équilibré (creditcard2023), les quatre algorithmes suivent le même schéma : division 80 %/20 % (train/test) stratifiée sur Class, appliquée à l'ensemble des 568 630 transactions, soit 454 904 transactions d'entraînement et 113 726 de test. Pour K-Means, un sous-ensemble de validation supplémentaire (10 % du train, soit 45 490 transactions) est en outre extrait.

Sur le dataset déséquilibré (creditcard), chaque algorithme opère sur un ensemble de taille différente :

- **Random Forest** : division 80 %/20 % de l'ensemble réel complet (284 807 transactions), soit **227 845** transactions d'entraînement et **56 962** de test, stratifiée sur Class. La validation des performances en entraînement s'appuie sur une validation croisée à 5 plis (*5-fold cross validation*), plutôt que sur un ensemble de validation distinct.
- **SVM** : division 80 %/20 % appliquée avant rééquilibrage, donnant les mêmes tailles que Random Forest (227 845/56 962). L'ensemble d'entraînement est ensuite réduit par sous-échantillonnage puis SMOTE (voir section 4.3), pour atteindre environ 20 433 transactions d'entraînement final (20 000 normaux + 394 fraudes, avant expansion SMOTE à ratio 0,1). La validation s'appuie sur une validation croisée à 3 plis.
- **RNN** : sur le sous-ensemble équilibré de 984 transactions (492 fraudes + 492 normales), division 80 %/20 % donnant 787 transactions d'entraînement et 197 de test. Un sous-ensemble de validation interne (20 % du train, géré directement par Keras via `validation_split=0.2` pendant l'entraînement) est utilisé pour l'arrêt anticipé.
- **LSTM** : sur le sous-ensemble équilibré plus large de 2 492 transactions (492 fraudes + 2 000 normales), division 80 %/20 % donnant 1 993 transactions d'entraînement et 499 de test, avec la même gestion de validation interne (20 % du train) que le RNN.
- **K-Means (et K-Means+PCA)** : sur le même sous-ensemble équilibré de 984 transactions, un schéma de division en trois parties **60 %/20 %/20 %** est appliqué (`train_test_split` utilisé en deux temps successifs), donnant environ 590 transactions d'entraînement, 197 de validation et 197 de test, toutes stratifiées sur Class.

2-6-2 Règle de décision et seuil de classification

Pour les modèles produisant une probabilité de fraude (Random Forest, SVM, RNN, LSTM), une transaction est classée comme frauduleuse si sa probabilité prédite dépasse un seuil de décision. Plutôt que d'utiliser systématiquement le seuil par défaut de 0,5, sept seuils candidats (0,1 à 0,7, par pas de 0,1) ont été testés pour chaque modèle concerné, et celui maximisant le F1-Score a été retenu :

- **Random Forest** (dataset déséquilibré) : seuil optimal de **0,3**, offrant un F1-Score de 88,3 % (contre 83,9 % au seuil par défaut de 0,5).
- **SVM** (dataset déséquilibré) : seuil optimal de **0,4**, offrant un F1-Score de 19,1 %.
- **RNN** (dataset déséquilibré) : seuil conservé à 0,5 (seuil par défaut), le sous-ensemble équilibré de test rendant l'optimisation du seuil moins déterminante.

Pour le **SVM**, la classification repose directement sur la position du point par rapport à l'hyperplan séparateur (fonction de décision du noyau RBF ou linéaire selon le dataset), sans notion explicite de seuil de probabilité.

Pour **K-Means**, la règle de décision diffère selon le dataset. Sur le **dataset déséquilibré**, la décision découle directement de l'appartenance au cluster préalablement identifié comme représentatif de la fraude, ce cluster étant celui présentant le taux de fraude moyen le plus élevé parmi les $K = 2$ clusters obtenus (classification binaire simple, sans seuil). Sur le **dataset équilibré**, une approche plus fine est utilisée : un score de fraude continu est calculé pour chaque transaction à partir des distances aux deux centroïdes ($\text{fraud_score} = \text{normal_dist} / (\text{normal_dist} + \text{fraud_dist})$), puis un seuil de décision est optimisé par recherche sur l'intervalle $[0,3; 0,8]$ (pas de 0,05) afin de maximiser le F1-Score sur l'ensemble de test, offrant un contrôle plus précis du compromis précision/rappel que la simple appartenance au cluster.

2-7 Conclusion

Ce chapitre a présenté l'approche proposée pour la détection des fraudes bancaires par analyse massive de données et intelligence artificielle. La démarche adoptée ainsi que le pipeline global du système ont d'abord été décrits en détail, algorithme par algorithme et dataset par dataset, mettant en évidence que le prétraitement effectif – normalisation, gestion du déséquilibre, division des données – diffère sensiblement d'une configuration à l'autre, plutôt que de suivre un schéma unique. La méthodologie de

chacun des quatre algorithmes retenus a ensuite été détaillée avec précision — Random Forest, SVM, K-Means et RNN (avec une variante LSTM sur le dataset déséquilibré) — en spécifiant pour chacun son architecture exacte, ses hyperparamètres réels et son adaptation propre aux spécificités de chaque dataset : trois mécanismes cumulés de gestion du déséquilibre pour le SVM (sous-échantillonnage, SMOTE, pondération des classes); une architecture de réseau récurrent qui varie selon le dataset pour le RNN; une règle de décision qui diffère entre appartenance simple au cluster et score de distance optimisé pour K-Means. Le processus d'extraction des caractéristiques a également été détaillé, incluant le nombre exact de variables reçues par chaque modèle et les transformations successives appliquées (mise à l'échelle séparée d'Amount et Time, réduction PCA pour K-Means+PCA). Enfin, la démarche

de classification a été définie avec précision, incluant les tailles exactes des ensembles d'entraînement, de validation et de test propres à chaque algorithme, les seuils de décision effectivement optimisés, ainsi qu'une clarification de ce que recouvre la notion de « validation » selon le modèle considéré (validation croisée, sous-ensemble interne, ou ensemble physiquement distinct). Cette méthodologie, exhaustivement documentée, constitue le socle expérimental sur lequel s'appuie le chapitre suivant, qui présente et analyse en détail les résultats obtenus par chaque modèle sur les deux datasets, avant de les comparer aux travaux connexes de la littérature.

Chapitre 3 : Résultats et Discussion

3-1 Introduction

Après avoir présenté dans le chapitre précédent l'architecture générale du système proposé ainsi que la méthodologie détaillée de chaque algorithme, ce chapitre est consacré à la mise en œuvre expérimentale et à l'analyse des résultats obtenus. L'environnement technique utilisé pour l'implémentation est d'abord détaillé, ainsi que les spécificités de chaque étape du pipeline expérimental. Les caractéristiques des deux datasets exploités sont ensuite rappelées, avant la présentation des métriques d'évaluation retenues, accompagnées de leurs formules mathématiques. L'essentiel de ce chapitre porte sur l'analyse des performances des quatre algorithmes — Random Forest, SVM, K-Means et RNN — évalués séparément sur le dataset déséquilibré et sur le dataset équilibré, avant une comparaison croisée permettant d'identifier l'impact réel du déséquilibre de classes sur chaque modèle. Enfin, les résultats obtenus sont situés par rapport aux travaux connexes identifiés dans l'état de l'art, afin de mettre en évidence les apports de ce mémoire par rapport à la littérature existante. Ce chapitre se termine par une synthèse des principaux enseignements tirés de cette étude comparative.

3-2 Configuration expérimentale

Cette section décrit l'environnement technique utilisé pour l'implémentation, ainsi que les détails pratiques de mise en œuvre de chaque étape du pipeline expérimental.

3-2-1 Environnement de travail

L'ensemble des expérimentations a été réalisé sur la plateforme **Kaggle Notebooks**, un environnement cloud gratuit fourni par Kaggle permettant d'exécuter du code Python dans des notebooks interactifs. Les deux datasets utilisés (`creditcard.csv` et `creditcard2023.csv`), initialement téléchargés en local, ont été importés manuellement sur la plateforme via la fonctionnalité d'upload de Kaggle, ce qui a permis d'y accéder directement dans l'environnement d'exécution sans dépendance à un stockage externe. Afin d'accélérer l'entraînement des modèles, notamment les réseaux de neurones récurrents (RNN et LSTM), l'accélérateur matériel **GPU T4 ×2** proposé par Kaggle a été activé, permettant de réduire significativement les temps de calcul par rapport à une exécution sur CPU. Le langage utilisé est **Python**, avec les principales bibliothèques suivantes :

Tableau 3.1 : Bibliothèques Python utilisées

Bibliothèque	Rôle
pandas / numpy	Manipulation et traitement des données tabulaires
scikit-learn	Random Forest, SVM, K-Means, prétraitement, métriques
imbalanced-learn (imblearn)	Application de la technique SMOTE
TensorFlow / Keras	Construction et entraînement des modèles RNN et LSTM
matplotlib / seaborn	Visualisation des résultats et matrices de confusion

3-2-2 Détails d'implémentation

Pour chaque algorithme, une démarche d'implémentation structurée en étapes reproductibles a été suivie :

- **Chargement et prétraitement** : lecture des fichiers CSV, vérification des valeurs manquantes, normalisation des variables Amount et Time avec `StandardScaler`.
- **Division des données** : séparation en ensembles d'entraînement (80 %) et de test (20 %), avec stratification sur la variable cible (Class), afin de préserver la même proportion de fraudes dans chaque sous-ensemble.
- **Gestion du déséquilibre** : application de SMOTE (`imblearn.over_sampling.SMOTE`) pour le SVM, sous-échantillonnage manuel pour K-Means et RNN, ou pondération des classes (`class_weight='balanced'`) pour Random Forest, selon l'algorithme.
- **Entraînement** : ajustement des hyperparamètres (détaillés au Chapitre 2, tableaux 2.1 à 2.4) et suivi de l'apprentissage via des callbacks Keras (`EarlyStopping`, `ReduceLROnPlateau`) pour les modèles RNN/LSTM, afin d'éviter le surapprentissage.
- **Optimisation du seuil de classification** : test de plusieurs seuils de décision (0,1 à 0,7) pour les modèles produisant une probabilité, avec sélection du seuil maximisant le F1-Score.
- **Évaluation** : calcul des métriques sur les ensembles d'entraînement, de validation et de test, génération des matrices de confusion et des rapports de classification (`classification_report` de scikit-learn).

Toutes les expérimentations ont été exécutées avec une graine aléatoire fixe (`random_state=42`) afin d'assurer la reproductibilité des résultats.

3-3 Base de données

Ce mémoire s'appuie sur deux datasets réels de transactions bancaires par carte de crédit, choisis afin d'étudier l'impact du déséquilibre de classes sur les performances des modèles de détection de fraude.

3-3-1 Dataset déséquilibré (creditcard.csv)

Ce dataset contient **284 807 transactions**, réparties en 31 variables : Time, V1 à V28 (composantes PCA), Amount et Class. La variable Time couvre une période de **48 heures** (2 jours) d'enregistrement. Il présente un déséquilibre de classes extrême, caractéristique des situations réelles de détection de fraude :

Tableau 3.2 : Répartition des classes – dataset déséquilibré

Classe	Nombre de transactions	Pourcentage
Normale (0)	284 315	99,83 %
Fraude (1)	492	0,17 %

Aucune valeur manquante n'a été détectée. On note en revanche la présence de **1 081 lignes dupliquées**, un point pris en compte lors du prétraitement. Le montant moyen des transactions (Amount) est de **88,35 €** (écart-type ≈ 250 €), avec un maximum de 25 691,16 €, ce qui traduit une forte dispersion des montants.

3-3-2 Dataset équilibré (creditcard2023.csv)

Ce second dataset contient **568 630 transactions**, réparties de manière strictement égale entre les deux classes :

Tableau 3.3 : Répartition des classes – dataset équilibré

Classe	Nombre de transactions	Pourcentage
Normale (0)	284 315	50 %
Fraude (1)	284 315	50 %

Il est intéressant de noter que le nombre de transactions normales dans ce dataset (284 315) correspond exactement au nombre de transactions normales du dataset creditcard.csv, ce qui suggère qu'il s'agit d'une version enrichie de fraudes synthétiques générées à partir des mêmes transactions légitimes. Ce dataset ne contient ni valeurs manquantes ni doublons. Il ne dispose pas de variable Time, mais conserve les variables V1 à V28 ainsi que Amount, dont le montant moyen est nettement plus élevé ($\approx 12\,041,96$ €), reflétant une distribution différente de celle du premier dataset.

Une première étape d'équilibrage du dataset déséquilibré a été réalisée à l'aide de la technique **SMOTE** (*Synthetic Minority Over-sampling Technique*), appliquée sur l'ensemble des 284 807 transactions afin de générer des exemples synthétiques de fraude et d'obtenir une distribution équilibrée (50 %/50 %) des deux classes.

Pour l'entraînement des modèles proprement dit, chaque algorithme met ensuite en œuvre sa

propre méthode d'équilibrage, adaptée à sa nature : pour **Random Forest**, aucune donnée n'est retirée ni synthétisée, le déséquilibre étant compensé uniquement par la pondération des classes (`class_weight='balanced'`) au niveau du classifieur ; pour le **SVM**, trois mécanismes sont cumulés : un sous-échantillonnage aléatoire réduit d'abord les transactions normales de l'ensemble d'entraînement à 20 000 exemples, puis la technique SMOTE génère des exemples synthétiques de fraude jusqu'à atteindre un ratio d'environ 10 %, et le classifieur est enfin configuré avec `class_weight='balanced'` ; pour **RNN et LSTM**, un sous-ensemble équilibré est constitué par simple tirage aléatoire des 492 fraudes complétées par un nombre égal (RNN) ou supérieur (LSTM) de transactions normales, sans SMOTE ; pour **K-Means** (et sa variante avec PCA), le même principe de tirage aléatoire équilibré est appliqué, la nature non supervisée de l'algorithme rendant cette approche suffisante. Le dataset `creditcard2023`, naturellement équilibré (50 %/50 %), n'a nécessité aucune technique de rééquilibrage pour aucun des quatre algorithmes.

3-3-3 Synthèse comparative

Tableau 3.4 : Synthèse comparative des deux datasets

Caractéristique	Dataset déséquilibré	Dataset équilibré
Nom du fichier	<code>creditcard.csv</code>	<code>creditcard2023.csv</code>
Nombre de transactions	284 807	568 630
Taux de fraude	0,17 %	50 %
Valeurs manquantes	Aucune	Aucune
Doublons	1 081	Aucun
Variable Time	Présente	Absente
Montant moyen (Amount)	88,35 €	12 041,96 €

Cette complémentarité entre les deux datasets permet d'évaluer les algorithmes à la fois dans des **conditions réalistes** (dataset déséquilibré) et dans des **conditions idéales** (dataset équilibré), offrant ainsi une lecture plus complète de leurs performances respectives.

Tableau 3.5 : Tailles des ensembles d'entraînement, de validation et de test

Algorithme	Train	Validation	Test	Total
Dataset déséquilibré (creditcard.csv)				
Random Forest	227 845	CV 5 plis	56 962	284 807
SVM	227 845 → 20 433 (après rééq.)	CV 3 plis	56 962	284 807
RNN	787	20 % du train (interne)	197	984
LSTM	1 993	20 % du train (interne)	499	2 492
K-Means (+ PCA)	590	197	197	984
Dataset équilibré (creditcard2023.csv)				
Random Forest	454 904	CV 5 plis	113 726	568 630
SVM	454 904	CV 5 plis	113 726	568 630
RNN	454 904	10 % du train (interne)	113 726	568 630
K-Means	454 904	45 490	113 726	568 630

La taille des ensembles utilisés varie fortement d'un algorithme à l'autre sur le dataset déséquilibré, en fonction de la stratégie de gestion du déséquilibre propre à chacun. Random Forest et SVM sont entraînés sur l'ensemble réel complet (227 845 transactions), avec une validation par validation croisée (5 plis pour Random Forest, 3 plis pour SVM) plutôt que par un ensemble distinct. À l'inverse, RNN, LSTM et K-Means sont entraînés sur des sous-ensembles nettement plus réduits (984 à 2 492 transactions), obtenus par tirage aléatoire équilibré, ce qui explique la taille très différente des ensembles de test correspondants (197 à 499 transactions, contre 56 962 pour Random Forest et SVM). Sur le dataset équilibré, les quatre algorithmes partagent le même schéma de division entraînement/test (80 %/20 %), appliqué à l'ensemble complet des 568 630 transactions. La validation y diffère cependant selon l'algorithme : Random Forest et SVM conservent une validation croisée à 5 plis, comme sur le dataset déséquilibré, tandis que le RNN utilise une validation interne Keras (10 % du train, gérée automatiquement pendant l'entraînement via `validation_split=0.1`) et K-Means bénéficie d'un ensemble de validation physiquement distinct de 45 490 transactions (10 % du train) pour le réglage du seuil de décision.

3-4 Métriques d'évaluation

Afin d'évaluer objectivement les performances des modèles de détection de fraude, plusieurs métriques complémentaires ont été utilisées. Toutes se basent sur les quatre éléments de la **matrice de confusion** : VP (Vrais Positifs), VN (Vrais Négatifs), FP (Faux Positifs) et FN (Faux Négatifs).

3-4-1 Accuracy (Exactitude)

L'Accuracy mesure la proportion globale de prédictions correctes :

$$Accuracy = \frac{VP + VN}{VP + VN + FP + FN} \quad (3.1)$$

Cette métrique, bien qu'intuitive, s'avère **trompeuse en contexte déséquilibré**, comme l'illustrent les résultats expérimentaux obtenus sur le dataset déséquilibré. Le modèle SVM (kernel RBF + SMOTE), évalué sur l'ensemble de test réel (56 962 transactions), atteint une Accuracy de 98,7 %, ce qui semble excellent à première vue. Pourtant, sa Précision sur la classe fraude n'est que de 10,7 % (706 fausses alertes pour seulement 85 fraudes correctement détectées), révélant un modèle en réalité peu fiable pour un usage opérationnel.

À l'inverse, le modèle K-Means, évalué sur un sous-ensemble équilibré de test (197 transactions), n'atteint qu'une Accuracy de 68,0 %, alors même que sa Précision sur la classe fraude est de 100 % (aucune fausse alerte) — il rate cependant 63 fraudes sur 98 (Recall de seulement 35,7 %). Le Random Forest, entraîné et testé sur les données réelles déséquilibrées, illustre quant à lui le comportement recherché : une Accuracy proche de 100 %, associée à une Précision de 92,2 % et un Recall de 84,7 %. Le RNN, évalué comme K-Means sur un sous-ensemble équilibré de 197 transactions, confirme cette tendance avec une Accuracy de 93,4 %, une Précision de 94,7 % et un Recall de 91,8 %.

Ces résultats montrent qu'une même valeur d'Accuracy peut correspondre à des comportements de modèle radicalement différents, et confirment la nécessité de s'appuyer sur des métriques complémentaires plutôt que sur l'Accuracy seule, en particulier lorsque la classe d'intérêt (la fraude) est fortement minoritaire.

3-4-2 Précision (Precision)

La Précision mesure, parmi les transactions prédites comme frauduleuses, la proportion qui l'est réellement :

$$Precision = \frac{VP}{VP + FP} \quad (3.2)$$

Une précision élevée indique peu de fausses alertes. Cette métrique est particulièrement révélatrice sur le dataset déséquilibré. Le SVM (kernel RBF + SMOTE), avec 706 faux positifs pour seulement 85 fraudes correctement détectées, n'atteint qu'une Précision de 10,7 % (révélant être de fausses alertes), ce qui rendrait le modèle inutilisable en production malgré son Accuracy de 98,7 % même dataset (aucun faux positif parmi ses prédictions de fraude), et Random Forest conserve une Précision élevée de 92,2 % équilibré, où la classe fraude n'est plus minoritaire, l'écart entre modèles se resserre nettement : la

Précision varie de 94,5 déséquilibre des classes, et non l’algorithme en lui-même, qui dégrade le plus fortement cette métrique pour le SVM.

3-4-3 Rappel (Recall)

Le Recall mesure, parmi les fraudes réelles, la proportion correctement détectée par le modèle :

$$Recall = \frac{VP}{VP + FN} \quad (3.3)$$

C’est la métrique la plus critique en détection de fraude : un Recall faible signifie que des fraudes passent inaperçues. C’est précisément la faiblesse de K-Means sur le dataset déséquilibré : malgré une Précision parfaite (100 — près des deux tiers des fraudes échappent au modèle. À l’inverse, Random Forest et RNN obtiennent sur le dataset équilibré un Recall de 100 laissant passer aucune fraude. Sur le dataset déséquilibré, le RNN atteint également un Recall élevé de 91,8 paradoxalement un Recall correct de 86,7 fraudes réelles tout en générant un nombre inacceptable de fausses alertes.

3-4-4 F1-Score

Le F1-Score est la moyenne harmonique entre Précision et Recall, offrant un compromis équilibré entre les deux :

$$F1-Score = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (3.4)$$

C’est la métrique de référence utilisée dans ce mémoire pour comparer les modèles, particulièrement adaptée aux jeux de données déséquilibrés. Son intérêt apparaît clairement dans le cas du SVM sur le dataset déséquilibré : bien que son Recall soit correct (86,7 fait chuter le F1-Score à seulement 19,1 une détection acceptable des fraudes. À l’opposé, Random Forest conserve un F1-Score élevé dans les deux configurations (88,3 entre Précision et Recall. Le RNN affiche le meilleur F1-Score du dataset déséquilibré parmi les modèles testés sur sous-ensemble équilibré (93,3 confirmant la hiérarchie déjà observée sur les métriques individuelles.

3-4-5 Taux de Faux Positifs (False Positive Rate)

Le FPR mesure la proportion de transactions normales classées à tort comme frauduleuses :

$$FPR = \frac{FP}{FP + VN} \quad (3.5)$$

Un FPR faible est essentiel en production, car un taux élevé de fausses alertes dégrade l’expérience utilisateur et surcharge les équipes de vérification. Sur le dataset déséquilibré, évalué sur l’ensemble de test réel complet (56 962 transactions), Random Forest obtient un FPR quasi nul (0,0 7 faux positifs sur 56 864 transactions normales), tandis que le SVM, avec 706 faux positifs sur ce même volume, atteint un FPR de 1,2 millions de transactions quotidiennes, se traduirait par un nombre de fausses alertes totalement ingérable pour les équipes de vérification. Sur le dataset équilibré, le RNN et Random Forest conservent les FPR les plus faibles (0,2 les fausses alertes même à grande échelle, alors que K-Means, avec un FPR de 5,0 moins performant des quatre modèles sur ce critère.

3-4-6 Silhouette Score (spécifique à K-Means)

Pour l'algorithme K-Means, la qualité de la séparation des clusters est en complément mesurée par le Silhouette Score, défini pour chaque point i par :

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \quad (3.6)$$

où $a(i)$ est la distance moyenne entre le point i et les autres points de son propre cluster, et $b(i)$ est la distance moyenne entre le point i et les points du cluster voisin le plus proche. Le score global correspond à la moyenne des $s(i)$ sur l'ensemble des points, et varie entre -1 (mauvaise séparation) et $+1$ (clusters bien séparés). Les résultats obtenus sur les deux datasets révèlent un constat à première vue contre-intuitif : le Silhouette Score est plus élevé sur le dataset déséquilibré (0,4389 sur le sous-ensemble de test équilibré à 984 échantillons) que sur le dataset équilibré (0,2824 sur les 113 726 transactions de test), alors même que le Recall y est nettement plus faible (35,7 86,0 compacité et la séparation des clusters dans l'espace des caractéristiques, indépendamment de leur concordance avec les étiquettes réelles fraude/normale : les deux clusters identifiés sur le dataset déséquilibré, bien que géométriquement mieux séparés, ne correspondent que partiellement à la frontière réelle entre transactions frauduleuses et légitimes, d'où le grand nombre de fraudes non détectées (63 sur 98) malgré un score de séparation supérieur. Sur le dataset équilibré, la réduction du Silhouette Score traduit une structure de données plus complexe où les deux classes se chevauchent davantage dans l'espace des caractéristiques, mais où l'algorithme parvient malgré tout à un meilleur alignement avec les étiquettes réelles. Ce constat illustre les limites intrinsèques d'une évaluation purement géométrique du clustering pour un problème de classification supervisée : le Silhouette Score reste un indicateur complémentaire utile, mais ne peut se substituer aux métriques de classification (Précision, Recall, F1-Score) pour juger de la pertinence opérationnelle d'un modèle de détection de fraude.

3-5 Analyse des performances

Cette section présente les résultats expérimentaux obtenus pour chacun des quatre algorithmes sur les deux datasets, avant de proposer une analyse comparative croisée.

3-5-1 Résultats sur le dataset déséquilibré (creditcard)

Le tableau suivant résume les performances des quatre algorithmes sur l'ensemble de test. Random Forest et SVM ont été évalués sur l'ensemble de test réel et complet (56 962 transactions), tandis que K-Means et RNN ont été évalués sur un sous-ensemble équilibré de test (197 transactions), en raison des contraintes liées à l'entraînement de ces modèles sur des données extrêmement déséquilibrées.

Tableau 3.5 : Résultats comparatifs sur le dataset déséquilibré (test)

Modèle	Accuracy	Précision	Recall	F1-Score	FPR
Random Forest	100,0 %	92,2 %	84,7 %	88,3 %	0,0 %
SVM (RBF + SMOTE)	98,7 %	10,7 %	86,7 %	19,1 %	1,2 %
K-Means	68,0 %	100,0 %	35,7 %	52,6 %	0,0 %
RNN	93,4 %	94,7 %	91,8 %	93,3 %	5,1 %
LSTM	96,8 %	93,7 %	89,9 %	91,8 %	1,5 %

Random Forest obtient le meilleur compromis sur les données réelles complètes, avec seulement 7 faux positifs et 15 fraudes manquées sur 56 962 transactions. Le SVM, malgré une Accuracy élevée, souffre d'une Précision très faible (706 fausses alertes pour 85 fraudes détectées), illustrant le piège de l'Accuracy en contexte déséquilibré déjà évoqué en section 4.1. K-Means, sur le sous-ensemble équilibré, atteint une Précision parfaite mais rate près de deux tiers des fraudes (Recall de 35,7 %). Le RNN, sur ce même type de sous-ensemble, présente l'équilibre le plus favorable entre Précision et Recall parmi les quatre modèles. Le LSTM, entraîné sur un sous-ensemble équilibré plus large (2 492 transactions), affiche l'Accuracy et le FPR les plus favorables de tous les modèles testés sur ce dataset (96,8 % et 1,5 % respectivement), mais avec un Recall légèrement inférieur à celui du RNN (89,9 % contre 91,8 %), ce qui traduit un compromis différent entre les deux architectures récurrentes.

Les matrices de confusion ci-dessous détaillent, pour chaque modèle, la répartition exacte des prédictions sur l'ensemble de test.

Tableau 3.6 : Matrice de confusion – Random Forest (dataset déséquilibré, 56 962 transactions)

	Prédit Normal	Prédit Fraude
Réel Normal	56 857	7
Réel Fraude	15	83

Sur les 56 962 transactions de l'ensemble de test, Random Forest classe correctement 56 940 d'entre elles

(99,96 %). Seules 7 transactions légitimes sont signalées à tort comme frauduleuses (faux positifs), un volume négligeable à l'échelle de la banque. Plus important encore, seules 15 fraudes sur 98 échappent au modèle (faux négatifs), soit un taux de fraudes manquées de 15,3 %. Ce résultat confirme que Random Forest parvient, grâce à la pondération des classes, à maintenir un contrôle simultané des deux types d'erreurs malgré l'extrême rareté de la classe fraude (0,17 % du dataset).

Tableau 3.7 : Matrice de confusion – SVM RBF + SMOTE (dataset déséquilibré, 56 962 transactions)

	Prédit Normal	Prédit Fraude
Réel Normal	56 158	706
Réel Fraude	13	85

La matrice de confusion du SVM illustre concrètement les limites déjà évoquées à la section 3-4-2 : sur 56 864 transactions normales, 706 sont signalées à tort comme frauduleuses, soit un volume de fausses alertes près de 100 fois supérieur à celui de Random Forest (7 faux positifs). En contrepartie, le modèle ne manque que 13 fraudes sur 98, un Recall correct qui masque cependant l'inutilisabilité opérationnelle du modèle : pour chaque fraude réellement détectée (85 au total), plus de 8 fausses alertes sont générées, un ratio incompatible avec un déploiement en production.

Tableau 3.8 : Matrice de confusion – K-Means (sous-ensemble équilibré de test, 197 transactions)

	Prédit Normal	Prédit Fraude
Réel Normal	99	0
Réel Fraude	63	35

Sur le sous-ensemble équilibré de test (197 transactions), K-Means ne produit aucun faux positif : les 99 transactions normales sont toutes correctement identifiées. Ce résultat, en apparence excellent, cache une faiblesse majeure du modèle : 63 fraudes sur 98 ne sont pas détectées (faux négatifs), soit près des deux tiers des cas frauduleux. Cette asymétrie extrême entre Précision (100 %) et Recall (35,7 %) illustre les limites d'une approche non supervisée qui privilégie la prudence au détriment de la détection effective des fraudes.

Tableau 3.9 : Matrice de confusion – RNN (sous-ensemble équilibré de test, 197 transactions)

	Prédit Normal	Prédit Fraude
Réel Normal	94	5
Réel Fraude	8	90

Sur ce même sous-ensemble équilibré de 197 transactions, le RNN affiche la matrice de confusion la plus équilibrée parmi les modèles évalués sur cette échelle réduite : seulement 5 faux positifs sur 99

transactions normales, et 8 fraudes manquées sur 98. Ce résultat traduit un compromis nettement plus favorable que K-Means, avec une différence de seulement 3 erreurs entre les deux types (5 faux positifs contre 8 faux négatifs), confirmant la capacité du réseau récurrent à apprendre une frontière de décision plus fine à partir d'un nombre limité d'exemples (787 transactions d'entraînement).

Tableau 3.10 : Matrice de confusion – LSTM (sous-ensemble équilibré de test, 499 transactions)

	Prédit Normal	Prédit Fraude
Réel Normal	394	6
Réel Fraude	10	89

Entraîné sur un sous-ensemble équilibré plus large (2 492 transactions), le LSTM est évalué ici sur 499 transactions de test. Il commet 6 faux positifs sur 400 transactions normales et 10 faux négatifs sur 99 fraudes, un résultat proche de celui du RNN en proportion mais obtenu sur un volume de test plus de deux fois supérieur (499 contre 197), renforçant la fiabilité statistique de cette évaluation. Le nombre de fraudes manquées (10) reste néanmoins supérieur en proportion à celui du RNN (8 sur 98), ce qui explique le Recall légèrement inférieur du LSTM (89,9 % contre 91,8 %) malgré une architecture plus profonde.

3-5-2 Résultats sur le dataset équilibré (creditcard2023)

Sur ce dataset, les quatre algorithmes ont été entraînés et évalués sur l'ensemble complet (113 726 transactions de test), sans nécessité de sous-échantillonnage :

Tableau 3.11 : Résultats comparatifs sur le dataset équilibré (test)

Modèle	Accuracy	Précision	Recall	F1-Score	FPR
Random Forest	99,9 %	99,9 %	100,0 %	99,9 %	0,1 %
SVM (Linéaire)	96,3 %	96,9 %	95,8 %	96,3 %	3,1 %
K-Means	90,5 %	94,5 %	86,0 %	90,1 %	5,0 %
RNN	99,9 %	99,8 %	100,0 %	99,9 %	0,2 %

Sur données équilibrées, Random Forest et RNN dominent nettement, avec un F1-Score quasi parfait (99,9 %) et un Recall de 100 % — aucune fraude n'échappe à ces deux modèles sur les 56 863 fraudes du test. Le SVM linéaire conserve de bonnes performances (F1 = 96,3 %), tandis que K-Means, bien qu'en net progrès par rapport au dataset déséquilibré, reste le modèle le moins performant du groupe, avec 7 934 fraudes non détectées sur 56 863 (Recall de 86,0 %) et un Silhouette Score modeste (0,2824), traduisant des clusters imparfaitement séparés.

Les matrices de confusion ci-dessous détaillent, pour chaque modèle, la répartition exacte des prédictions sur l'ensemble de test.

Tableau 3.12 : Matrice de confusion – Random Forest (dataset équilibré, 113 726 transactions)

	Prédit Normal	Prédit Fraude
Réel Normal	56 784	79
Réel Fraude	0	56 863

Sur les 113 726 transactions de test, Random Forest atteint un résultat quasi parfait : les 56 863 fraudes réelles sont toutes détectées, sans aucun faux négatif. Seules 79 transactions légitimes sur 56 863 sont signalées à tort comme frauduleuses, un volume qui reste marginal (FPR de 0,1 %). Ce résultat confirme que, dans des conditions de classes équilibrées, Random Forest atteint la limite haute de ce qu'un modèle supervisé peut offrir pour ce problème.

Tableau 3.13 : Matrice de confusion – SVM Linéaire (dataset équilibré, 113 726 transactions)

	Prédit Normal	Prédit Fraude
Réel Normal	55 116	1 747
Réel Fraude	2 406	54 457

Le SVM linéaire, bien que nettement plus performant sur ce dataset équilibré que sur le dataset déséquilibré, laisse échapper 2 406 fraudes sur 56 863 (Recall de 95,8 %) et génère 1 747 faux positifs sur 56 863 transactions normales. Ces deux volumes d'erreurs, bien supérieurs à ceux de Random Forest (0 faux négatif, 79 faux positifs), illustrent que même dans des conditions idéales de classes équilibrées, le choix de l'algorithme continue d'influencer significativement la qualité de la détection.

Tableau 3.14 : Matrice de confusion – K-Means (dataset équilibré, 113 726 transactions)

	Prédit Normal	Prédit Fraude
Réel Normal	54 005	2 858
Réel Fraude	7 934	48 929

La matrice de confusion de K-Means sur le dataset équilibré confirme qu'il s'agit du modèle le moins performant du groupe : 7 934 fraudes sur 56 863 ne sont pas détectées, soit près de 14 % des cas frauduleux, un volume nettement supérieur à celui des trois modèles supervisés. Le nombre de faux positifs (2 858) est également le plus élevé des quatre modèles testés sur ce dataset. Ces deux résultats confirment que la nature non supervisée de l'algorithme, qui ne s'appuie que sur la distance aux centroïdes sans apprentissage direct de la frontière fraude/normale, limite structurellement sa capacité de discrimination, y compris lorsque les classes sont équilibrées.

Tableau 3.15 : Matrice de confusion – RNN (dataset équilibré, 113 726 transactions)

	Prédit Normal	Prédit Fraude
Réel Normal	56 761	102
Réel Fraude	0	56 863

Comme Random Forest, le RNN détecte l'intégralité des 56 863 fraudes du test, sans aucun faux négatif. Il génère toutefois davantage de faux positifs que Random Forest (102 contre 79 sur 56 863 transactions normales), ce qui explique son FPR légèrement supérieur (0,2 % contre 0,1 %) malgré un F1-Score identique (99,9 %) entre les deux modèles. Ce résultat confirme que Random Forest et RNN constituent, sur données équilibrées, les deux modèles les plus fiables pour ne manquer aucune fraude réelle.

3-5-3 Comparaison croisée et discussion

La comparaison des deux configurations met en évidence plusieurs enseignements :

- **Random Forest** est le modèle le plus robuste face au déséquilibre : ses performances restent élevées dans les deux configurations (F1 de 88,3 % à 99,9 %), grâce notamment à la pondération des classes (`class_weight='balanced'`) et à l'agrégation de multiples arbres.
- **SVM** est le modèle le plus sensible au déséquilibre : son F1-Score chute de 96,3 % (équilibré) à seulement 19,1 % (déséquilibré), la Précision sur la classe fraude s'effondrant à 10,7 % malgré l'application de SMOTE.
- **K-Means**, de par sa nature non supervisée, obtient les performances les plus faibles dans les deux configurations, avec un Recall systématiquement inférieur aux modèles supervisés, confirmant les limites d'une approche par clustering pour ce problème.
- **RNN** affiche une bonne robustesse relative (F1 de 93,3 % à 99,9 %), bien que les résultats sur le dataset déséquilibré aient été obtenus sur un sous-ensemble équilibré et non sur les données réelles complètes, ce qui appelle à la prudence dans l'interprétation directe de cette robustesse.
- **LSTM**, testé uniquement sur le dataset déséquilibré, se distingue par le meilleur FPR (1,5 %) et la meilleure Accuracy (96,8 %) de tous les modèles évalués sur ce dataset, au prix d'un Recall légèrement plus faible que le RNN (89,9 % contre 91,8 %). Entraîné sur un sous-ensemble équilibré plus large (2 492 échantillons contre 984 pour le RNN), il illustre qu'une architecture plus profonde et davantage de données d'entraînement ne se traduisent pas automatiquement par un meilleur Recall, ce dernier restant la métrique la plus critique en détection de fraude.

Ces résultats confirment la problématique posée en introduction de ce mémoire : le déséquilibre de classes affecte significativement les performances des modèles de Machine Learning, mais de manière très inégale selon l'algorithme utilisé, Random Forest se distinguant comme le choix le plus fiable pour un déploiement sur données réelles déséquilibrées.

3-6 Étude comparative avec les travaux connexes

Afin de situer les résultats de ce mémoire par rapport à la littérature récente, cette section les compare à deux travaux publiés en 2025 traitant de la même problématique (détection de fraude bancaire), s'appuyant sur le même dataset ULB *Credit Card Fraud Detection* (284 807 transactions, 492 fraudes).

Tableau 3.16 : Comparaison avec les travaux connexes (même dataset)

Référence	Année	Algorithme	Test set	Précision	Recall	F1
Popova & Gardi [22]	2025	Random Forest	Réel complet	94,2 %	78,9 %	85,8 %
Zhou et al. [23]	2025	RNN-LSTM hybride	Réel complet	90,6 %	80,6 %	85,3 %
Notre approche	2026	Random Forest	Réel complet	92,2 %	84,7 %	88,3 %
Notre approche	2026	RNN	Équilibré (197)	94,7 %	91,8 %	93,3 %
Notre approche	2026	LSTM	Équilibré (499)	93,7 %	89,9 %	91,8 %

Popova & Gardi (2025) évaluent leur Random Forest sur l'ensemble de test réel complet, comme dans ce mémoire. Le modèle développé dans ce mémoire obtient un F1-Score supérieur (88,3 % contre 85,8 %) et un Recall supérieur (84,7 % contre 78,9 %), ce qui peut s'expliquer par l'utilisation combinée de la pondération des classes et de l'optimisation du seuil de classification. **Zhou et al. (2025)** obtiennent un F1-Score de 85,3 % avec leur modèle RNN-LSTM hybride sur l'ensemble de test réel complet. Le RNN développé dans ce mémoire atteint un F1-Score de 93,3 %, supérieur à leur résultat ; cette différence doit cependant être interprétée avec prudence, ce modèle ayant été évalué sur un sous-ensemble équilibré (197 transactions) et non sur l'ensemble complet — une limite identifiée de ce mémoire. Le LSTM développé dans ce mémoire, architecture plus proche du modèle hybride de Zhou et al., atteint quant à lui un F1-Score de 91,8 %, également supérieur à leur résultat, avec la même réserve méthodologique concernant la taille et la nature du sous-ensemble de test (499 transactions, contre l'ensemble réel complet chez Zhou et al.). **Limite de la comparaison** : cette étude porte uniquement sur Random Forest, RNN et LSTM, seuls algorithmes communs entre ce mémoire et les deux articles retenus, ni SVM ni K-Means n'y étant testés. **Constat général** : le Random Forest développé dans ce mémoire est compétitif, et même légèrement supérieur, à des travaux très récents (2025) traitant du même problème avec le même dataset — confirmant la pertinence de la pondération des classes et de l'optimisation du seuil, deux pratiques rigoureusement appliquées dans ce mémoire.

3-7 Conclusion

Ce chapitre a permis de présenter et d’analyser en détail les résultats expérimentaux obtenus par les quatre algorithmes étudiés — Random Forest, SVM, K-Means et RNN (ainsi que sa variante LSTM sur le dataset déséquilibré) — sur les deux configurations de données de ce mémoire. L’analyse des

performances a confirmé la problématique posée en introduction de ce mémoire : le déséquilibre de classes affecte significativement les modèles de Machine Learning, mais de manière très inégale selon l’algorithme utilisé. Random Forest s’est révélé le modèle le plus robuste, conservant de bonnes performances dans les deux configurations (F1 de 88,3 % à 99,9 %), tandis que SVM s’est montré le plus sensible au déséquilibre, avec un effondrement de sa Précision malgré l’application de SMOTE. K-Means, de par sa nature non supervisée, a confirmé ses limites face à un problème de classification binaire complexe, et RNN a affiché une robustesse relative, bien que son évaluation sur le dataset déséquilibré mériterait d’être approfondie sur l’ensemble de test réel complet. Le LSTM, testé uniquement sur le dataset déséquilibré, a obtenu le meilleur FPR (1,5 %) et la meilleure Accuracy (96,8 %) parmi les modèles évalués sur ce dataset, confirmant l’intérêt des architectures récurrentes profondes pour ce problème, tout en affichant un Recall légèrement inférieur à celui du RNN simple. La confrontation de ces résultats à deux travaux récents (2025) traitant de la même

problématique sur le même dataset a permis de situer objectivement ce mémoire dans le paysage scientifique actuel : le modèle Random Forest développé dans ce mémoire s’est révélé compétitif, voire légèrement supérieur, à des approches comparables publiées récemment, notamment grâce à la pondération des classes et à l’optimisation du seuil de classification. Ces résultats confirment que

Random Forest constitue le choix le plus fiable pour un déploiement opérationnel de détection de fraude sur données réelles déséquilibrées, tandis que les approches par Deep Learning (RNN/LSTM) représentent une piste prometteuse nécessitant des données d’entraînement plus abondantes et une évaluation plus rigoureuse sur données réelles complètes. Le chapitre suivant, consacré à la Conclusion Générale et aux Perspectives, synthétise l’ensemble de la démarche menée dans ce mémoire et propose des pistes d’amélioration pour les travaux futurs.

Conclusion Générale et Perspectives

Ce mémoire visait à étudier la détection des fraudes bancaires par analyse massive de données et intelligence artificielle, et plus précisément à mesurer l'impact du déséquilibre de classes sur les performances des algorithmes de Machine Learning et de Deep Learning.

Le premier chapitre a permis de dresser un état de l'art sur la fraude bancaire : ses différentes typologies, mais aussi les méthodes de détection existantes, qu'il s'agisse des approches traditionnelles basées sur des règles fixes ou des techniques plus récentes issues du Machine Learning et du Deep Learning. La confrontation avec trois travaux connexes (Hemanth Kumar et al., Popova & Gardi, Zhou et al.) a également permis de vérifier que les algorithmes retenus pour ce mémoire restaient pertinents au regard de la littérature actuelle.

Le deuxième chapitre s'est ensuite attaché à décrire l'approche développée : l'architecture générale du système, la méthodologie propre à chaque algorithme (Random Forest, SVM, K-Means, RNN), l'extraction des caractéristiques à partir des données transactionnelles, et enfin la démarche suivie pour la classification et l'évaluation des modèles.

Le troisième chapitre a présenté les résultats obtenus sur les deux datasets étudiés — l'un réel et déséquilibré, l'autre équilibré — en les mettant en perspective avec la littérature récente. Il en ressort que **Random Forest** constitue le modèle le plus robuste face au déséquilibre de classes, avec des performances qui restent bonnes dans les deux configurations (F1-Score de 88,3 % à 99,9 %). Le **SVM**, lui, s'est montré nettement plus sensible à ce déséquilibre : sa Précision sur la classe fraude s'effondre malgré l'application de SMOTE. De son côté, **K-Means**, en tant qu'algorithme non supervisé, a mis en évidence les limites d'une approche par clustering pour ce type de problème de classification binaire. Le **RNN**, quant à lui, affiche des résultats prometteurs qui mériteraient d'être approfondis sur des données réelles complètes.

La comparaison avec deux travaux publiés en 2025 sur ce même dataset a par ailleurs confirmé que l'approche développée dans ce mémoire tient la comparaison, notamment grâce à la pondération des classes et à l'optimisation du seuil de classification, deux choix qui se sont avérés déterminants pour mieux détecter les fraudes sans multiplier les fausses alertes.

Ce travail comporte cependant certaines limites. L'évaluation du RNN et de K-Means sur le dataset déséquilibré a été menée sur un sous-ensemble équilibré de test (197 transactions), et non sur l'ensemble complet des données réelles ; les résultats obtenus pour ces deux modèles ne sont donc pas directement comparables à ceux de Random Forest et SVM. Les hyperparamètres ont par ailleurs été choisis de manière empirique plutôt qu'à l'issue d'une recherche exhaustive de type grid search ou optimisation

bayésienne, ce qui laisse sans doute une marge de progression pour chaque algorithme. Les deux datasets utilisés, bien que réels, restent également limités au contexte européen des transactions par carte bancaire et à une fenêtre temporelle assez courte (48 heures pour le dataset déséquilibré), ce qui peut restreindre la portée des résultats à d'autres contextes géographiques ou temporels. Ces limites tracent naturellement plusieurs pistes pour la suite : il serait utile de réévaluer le RNN et K-Means sur l'ensemble de test réel et complet des deux datasets, afin de pouvoir les comparer plus rigoureusement à Random Forest, à SVM, et aux résultats de Zhou et al. (2025). Une recherche systématique des hyperparamètres, par grid search ou optimisation bayésienne, pourrait aussi améliorer encore les performances obtenues. On pourrait également explorer des architectures de Deep Learning plus récentes, comme les Transformers ou les mécanismes d'attention, qui offrent des perspectives intéressantes pour la détection de fraude sur données séquentielles. Enfin, un déploiement en conditions réelles supposerait d'étudier plus en détail la capacité du système à s'adapter en continu à l'évolution des techniques de fraude, ainsi que sa robustesse face à d'éventuelles tentatives d'évasion adverses.

En définitive, ce mémoire montre que l'intelligence artificielle et l'analyse massive de données constituent des leviers efficaces pour détecter automatiquement la fraude bancaire, à condition de bien choisir l'algorithme et la méthodologie d'évaluation en fonction des contraintes réelles posées par le déséquilibre des classes.

Bibliographie

- [1] Dictionnaire Juridique, *Définition de Fraude*, [En ligne]. Disponible sur : <https://www.dictionnaire-juridique.com/definition/fraude.php>.
- [2] Préfecture de la Martinique, *Les types de fraude*, [En ligne]. Disponible sur : <https://www.martinique.gouv.fr/Actions-de-l-Etat/Securite/Securite-des-personnes-et-des-biens/Les-types-de-fraude>.
- [3] FIPECO, *La fraude et le contrôle fiscal*, [En ligne]. Disponible sur : <https://www.fipeco.fr/fiche/La-fraude-et-le-contr%C3%B4le-fiscal>.
- [4] Cabinet ACI, *Infractions douanières : définition, sanctions et défense pénale*, [En ligne]. Disponible sur : <https://www.cabinetaci.com/infractions-douanieres-definition-sanctions-et-defense-penale/>.
- [5] Allianz Trade, *Qu'est-ce que la fraude interne?*, [En ligne]. Disponible sur : <https://www.allianz-trade.fr/blog/qu-est-ce-que-la-fraude-interne.html>.
- [6] Les Clés de la Banque, *Fraudes courantes*, Fédération Bancaire Française, [En ligne]. Disponible sur : <https://www.lesclesdelabanque.com/particulier/fraudes-courantes/>
- [7] SAS, *Détection de la fraude et Machine Learning*, [En ligne]. Disponible sur : https://www.sas.com/fr_fr/insights/articles/risk-fraud/fraud-detection-machine-learning.html
- [8] DataDome, *Comment améliorer la détection de la fraude avec l'apprentissage automatique*, 2023, [En ligne]. Disponible sur : <https://datadome.co/fr/learning-center/comment-ameliorer-la-detection-de-la-fraude-avec-le-machine-learning/>
- [9] Entrust, *PSD2*, [En ligne]. Disponible sur : <https://www.entrust.com/fr/resources/learn/psd2>.
- [10] Stripe, *Qu'est-ce que la DSP2? Ce que les entreprises doivent savoir*, [En ligne]. Disponible sur : <https://stripe.com/fr/resources/more/what-is-psd2-here-is-what-businesses-need-to-know>.
- [11] DataDome, *RGPD et prévention de la fraude : impact des réglementations de conformité*, [En ligne]. Disponible sur : <https://datadome.co/fr/learning-center/rgpd-et-prevention-de-la-fraude-impact-des-reglementations-de-conformite/>.
- [12] Data Bird, *IA et cybersécurité*, [En ligne]. Disponible sur : <https://www.data-bird.co/blog/ia-cybersecurite>.
- [13] DataDome, *Détection de la fraude par IA dans le secteur bancaire*, 2025, [En ligne]. Disponible sur : <https://datadome.co/fr/learning-center/fraude-ia-banque/>

- [14] Management & Data Science, *Comment le machine learning permet de détecter la fraude bancaire?*, 2021, [En ligne]. Disponible sur : <https://management-datascience.org/articles/16671/>
- [15] Banque de France, *Note sur la fraude aux moyens de paiement – 1er semestre 2022*, Observatoire de la sécurité des moyens de paiement, 2023, [En ligne]. Disponible sur : https://www.banque-france.fr/system/files/2023-08/Banque_de_France-Strategie-monetaire-20230327_osmp_note-fraude-1er-semestre-2022.pdf
- [16] Trend Micro, *Qu'est-ce que le phishing ?*, [En ligne]. Disponible sur : https://www.trendmicro.com/fr_fr/what-is/phishing.html
- [17] IJRASET, *Credit Card Fraud Detection using Random Forest Algorithm*, International Journal for Research in Applied Science and Engineering Technology, [En ligne]. Disponible sur : <https://www.ijraset.com/research-paper/credit-card-fraud-detection-using-random-forest-algorithm>
- [18] ResearchGate, *Principe des SVM : Recherche de l'hyperplan optimal*, [En ligne]. Disponible sur : https://www.researchgate.net/figure/Principe-des-SVM-a-Recherche-de-lhyperplan-optimal-b-Hyperplan-optimal-marge-et_fig7_40268349
- [19] DellData, *K-Means : l'algorithme de clustering*, [En ligne]. Disponible sur : <https://delladata.fr/kmeans/>
- [20] Studyopedia, *Recurrent Neural Networks (RNN)*, [En ligne]. Disponible sur : <https://studyopedia.com/deep-learning/recurrent-neural-networks/>
- [21] B. Hemanth Kumar, S. T. Nuka, M. Malempati, H. K. Sriram, S. Mashetty, S. Kannan, *Big Data in Cybersecurity : Enhancing Threat Detection with AI and ML*, Metallurgical and Materials Engineering, Vol. 31(3), 2025, pp. 12–20. [En ligne]. Disponible sur : <https://doi.org/10.63278/1315>
- [22] I. Popova, H. A. A. Gardi, *Credit Card Fraud Detection*, arXiv preprint arXiv :2509.15044, 2025. [En ligne]. Disponible sur : <https://arxiv.org/abs/2509.15044>
- [23] H. Zhou, X. Zheng, J. Du, *A Deep Learning Hybrid RNN-LSTM Model for Credit Card Fraud Detection*, preprint, Xi'an University of Technology, 2025. [En ligne]. Disponible sur : <https://doi.org/10.21203/rs.3.rs-8327749/v1>